Graph-Structured Memory for Cognitive Agents: From Knowledge Graphs to Geometric Representations

Michael Banf, Johannes Kuhn, Slawomir Garcarz, Liliya Imasheva, Dominik Filipiak, Marc Stiller, Gustaw Kalek, Maximilian Schattauer, Anton Steuer, Gregorz Luczyna, Konstantin Gärtner and Max Becker

Perelyn GmbH, Munich, Germany

Abstract

LLM-based agents require persistent memory that spans sessions, tracks temporal evolution, and supports multi-hop reasoning—capabilities that flat memory stores (vector retrieval, key-value extraction) cannot provide. Graph-structured memory emerges as the dominant paradigm: across the 10 capability dimensions cognitive agents need, graph+LLM systems cover 8–9 while flat approaches cover at most 3, and graph memory disproportionately benefits smaller models (up to 17× stronger improvement at 7B than at frontier scale). We present a survey synthesizing 160+ papers (2023–2026), identifying six architectural patterns, a retrieval spectrum from training-free graph algorithms through reinforcement learning to GNN message passing, a memory evolution taxonomy, neuroscience-grounded consolidation mechanisms, and security vulnerabilities in current designs. We map open challenges onto *geometric graph representations*—discrete Ricci curvature, hyperbolic embeddings, Lie group manifolds, and continuous-time dynamics—as a frontier where adjacent results demonstrate concrete gains but no agent memory system has yet exploited.

Keywords

Knowledge Graphs, Agent Memory, Temporal Knowledge Graphs, Graph Neural Networks, Geometric Graph Representations, LLM Agents

1. Introduction

The dominant memory pattern for LLM agents is stateless: each session begins from scratch. When persistence is added, it typically takes the form of flat extraction—triplet stores, key-value memories, or embedding-indexed chunks [1, 2]. These approaches handle factual recall but fail systematically on temporal reasoning, multi-hop inference, and knowledge evolution.

Practitioners are already responding to this limitation. Karpathy's *LLM Wiki* pattern [174] replaces per-query RAG with an LLM that incrementally builds and maintains an interlinked wiki of entity pages—pre-compiling knowledge into a persistent graph of cross-referenced markdown files so that answers *compound over time* rather than being rediscovered from scratch. Implementations range from Obsidian-based agents that maintain knowledge graphs via wikilinks [175, 176] to the open-source Khoj [177] "AI second brain" with semantic search over personal document collections, and the LLM Wiki desktop application [178] with Louvain community detection and a 4-signal knowledge graph. These tools demonstrate real-world demand for graph-structured persistent memory, yet they remain engineering artifacts without principled architectures for temporal reasoning, consolidation, or geometric representations—precisely the gaps this survey addresses.

Table 1 makes the structural argument explicit. Flat memory paradigms—vector stores, key-value extraction, and even the emerging LLM Wiki pattern—cover at most 3–4 of the 10 capability dimensions that cognitive agent memory requires. Graph+LLM systems, by contrast, routinely cover 8–9: multi-hop traversal, temporal ordering, causal structure, hierarchical consolidation, multi-agent sharing, and learned retrieval all emerge naturally from graph representations. The only dimension where flat approaches have an advantage is raw scalability—and even this gap is narrowing as graph databases reach sub-140 ms p99 latency in production [15]. Our survey of 30 graph-memory systems (§3) and

FGWM 2026: Workshop of the GI Special Interest Group on Knowledge Management, August 11–12, 2026, Bremen, Germany
michael.banf@perelyn.com (M. Banf)

Table 1
Why graphs? Capability comparison across memory paradigms. ✓ = supported, △ = partial, — = absent. Based on the 30 graph-memory systems in our corpus (Table 3) versus flat baselines (Mem0 vector-only, MemGPT, vanilla RAG). Graph+LLM systems cover 7–9 of 10 dimensions; flat systems cover at most 3.

Capability	Vector/ RAG	KV/ Triplet	LLM Wiki (Obsidian)	Graph+ LLM
Multi-hop reasoning	—	—	△	✓
Temporal ordering	—	—	—	✓
Validity intervals	—	—	—	△
Causal structure	—	—	—	✓
Consolidation	—	—	△	✓
Hierarchical structure	—	—	△	✓
Incremental update	✓	✓	✓	✓
Scalability	✓	✓	△	△
Multi-agent sharing	△	—	—	✓
RL/learned retrieval	—	—	—	✓
Dimensions covered	**2**	**1**	**3–4**	**8–9**

7 open-source libraries (Table 7) confirms that *graph+LLM is the only paradigm where a single system can address all dimensions simultaneously*: OpenMemory is the sole library supporting graph storage, retrieval, temporal mechanisms, hierarchy, and agent integration.

Benchmark evidence quantifies the deficit. LoCoMo-Plus [3] reports cognitive accuracy below 25% across Mem0, Letta, and Zep, with a 35–45 percentage-point drop from factual to cognitive tasks. MemoryAgentBench [4] finds all methods below 7% on multi-hop conflict resolution. On LongMemEval [5], multi-strategy systems combining graph traversal with vector and keyword search achieve 91.4% retrieval accuracy, compared to 49.0% for extraction-based systems—but at 10–20× the latency [6]. Flat memory architectures cannot represent the relational, temporal, and hierarchical structure that cognitive tasks demand.

Graph-based memory addresses this by making structure explicit. Knowledge graphs ground entities and relations; temporal KGs add validity intervals; hierarchical event graphs capture episodic consolidation. To quantify the landscape, we assembled a corpus of 68 papers from the Awesome-GraphMemory collection [7], of which 30 constitute actual graph-based agent memory systems. Our analysis of all 70 papers from the ICLR 2026 MemAgents workshop [8] finds that 8 (11%) employ graph structures—the second-largest method category after retrieval-augmented approaches. This complements concurrent surveys: Hu et al. [131] provide a comprehensive taxonomy of agent memory but focus on the "when" and "how" of memory operations rather than graph structure; Chen et al. [132] bridge cognitive neuroscience and agent memory design, synthesizing interdisciplinary insights on memory systems from working memory to consolidation; a concurrent evolutionary framework [159] traces memory from Storage (trajectory preservation) through Reflection (trajectory refinement) to Experience (trajectory abstraction), identifying proactive exploration and cross-trajectory synthesis as frontier mechanisms.

Three findings stand out. First, entity-relation KGs account for 60% of graph-memory papers; non-KG structures remain rare. Second, 57% implement no temporal mechanism—a field-wide deficit, not a workshop artifact. Third, retrieval is overwhelmingly training-free: graph algorithms (PPR, Leiden, recency decay) and multi-strategy heuristics power every production system (Graphiti, HippoRAG, Cognee, FalkorDB+Mem0, Hindsight), while only 2 of 30 papers apply GNN message passing.

Figure 1 illustrates the structural progression from flat memory to geometric representations, with each level adding expressivity at the cost of complexity.

This paper contributes: (1) a taxonomy of graph-based agent memory architectures grounded in the corpus (§3), with a comprehensive 24-system comparison table; (2) an analysis of temporal KG approaches including neuroscience-inspired mechanisms (§4); (3) a retrieval spectrum from training-free graph algorithms through post-retrieval verification to learned approaches (§5); (4) a model-size equalizer analysis showing graph memory disproportionately benefits smaller models (§5.7); (5) a memory evolution taxonomy distinguishing internal self-evolving from external exploration (§6); (6) a mapping of geometric graph representations—hypergraphs, discrete curvature, hyperbolic embeddings, Lie

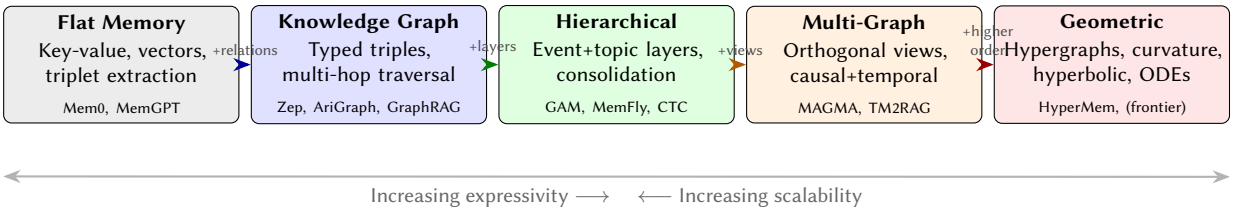

Figure 1: Structural progression of agent memory representations. Each level adds expressivity—relations, hierarchical layers, orthogonal views, higher-order structure—at the cost of construction and retrieval complexity. Current production systems operate at the KG and hierarchical levels; geometric representations remain a largely unexplored frontier (§8).

group manifolds, and continuous-time dynamics—onto open memory challenges (§8); (7) a benchmark landscape and open-source ecosystem survey; and (8) a research agenda identifying six open problems (§9).

2. Background: The Agent Memory Problem

The CoALA taxonomy [11] maps human memory onto agent components: *working memory* (the context window), *semantic memory* (factual knowledge), *episodic memory* (past experiences), and *procedural memory* (learned skills). A finer-grained survey [12] refines this into three axes—*forms*, *functions*, and *dynamics*—which better captures implementation-level distinctions. We use CoALA as our primary lens because its categories map cleanly onto graph-structure choices.

The survey [7] formalizes graph-based memory as a dynamic attributed graph $\mathcal{M}_G = G_t = (V_t, E_t, X_t)$ where memory units (events, entities, concepts) are nodes, and semantic, temporal, causal, or logical relationships are edges. The lifecycle $\mathcal{L}$ consists of four stages: *extraction* (transforming raw observations o_t into structured memory units m), *storage* (organizing units via indexing and structural organization), *retrieval* (selecting $\mathcal{M}_{rel} \subset \mathcal{M}$ in response to query q), and *evolution* (refining memory through internal self-evolving or external self-exploration). Four atomic operations manipulate $\mathcal{M}$: $\text{Write}(m, \mathcal{M}) \to \mathcal{M}'$, $\text{Read}(q, \mathcal{M}) \to \mathcal{M}_q$, $\text{Update}(m, \mathcal{M}) \to \mathcal{M}'$, and $\text{Delete}(m, \mathcal{M}) \to \mathcal{M}'$.

Our quantitative analysis of the MemAgents workshop reveals skewed research effort: episodic memory appears in 59% of papers and semantic in 56%, yet procedural memory in only 24%. Operations are similarly imbalanced: 84% address retrieval and 79% writing, but only 31% tackle consolidation, 23% handle updates, and 19% implement forgetting.

The *episodic-temporal paradox* is particularly striking: 59% address episodic memory, yet only 4% implement temporal mechanisms beyond basic timestamps. Episodes are treated as unordered collections rather than temporally situated experiences. Graphs naturally encode temporal ordering (directed edges), causal dependencies (DAGs), validity intervals (edge attributes), and hierarchical consolidation (graph coarsening).

Production systems demonstrate the practical value: **Graphiti/Zep** [14] implements a bi-temporal KG with validity intervals, achieving 94.8% on Deep Memory Retrieval. **HippoRAG** [9] models hippocampal indexing as an open KG with Personalized PageRank, outperforming embedding retrieval by 3–10 F1 on multi-hop benchmarks. **Cognee** builds KGs via an ECL pipeline with explicit memory decay, deployed at over 70 companies. **FalkorDB+Mem0** [15] replaces Mem0's vector store with a graph database via runtime Cypher translation, achieving sub-140 ms p99 with per-user graph isolation enabling GDPR-compliant deletion. **Hindsight** [16] runs four parallel retrieval pathways (semantic, BM25, graph traversal, temporal filtering) with cross-encoder re-ranking, achieving 91.4% on LongMemEval. Neo4j's agent-memory library [17] implements a three-memory architecture (conversation history, entity KG, reasoning traces with causal provenance) using the POLE+O ontology—distinguishing *context graphs* that capture dynamic agent state from static knowledge graphs.

Table 2
Benchmark landscape for agent memory evaluation, organized by scenario type. Count indicates number of distinct benchmarks identified in the survey [7].

Scenario	Count	Representative Benchmarks
Interaction	15	LoCoMo, LongMemEval, MemAgentBench
Long-context	12	HotpotQA, Musique, BABILong
Environments	8	ALFWorld, ScienceWorld, AgentBoard
Personalization	6	PersonaMem, PerLTQA, LOCCO
Web agents	5	WebArena, MT-Mind2Web
Tool/general	5	SWE-Bench, GAIA, ToolBench
Continual	4	MemoryBench, StreamBench

2.1. Benchmark Landscape

The evaluation landscape for graph-based agent memory is fragmented across seven distinct scenarios (Table 2). The survey by Yang et al. [7] catalogs over 50 benchmarks, yet most measure downstream task accuracy rather than memory quality itself.

Interaction benchmarks (LoCoMo, LongMemEval, MemoryAgentBench, MEMTRACK, MADial-Bench) test long-range context and consistency across sessions; however, many lack explicit supervision for memory updates when dealing with conflicting facts. *Long-context* benchmarks (HotpotQA, Musique, BABILong, RULER) test evidence aggregation and multi-step reasoning but often remain single-turn and do not require persistent memory stores. *Continual* benchmarks (MemoryBench, LifelongAgentBench, StreamBench, Evo-Memory) represent lifelong memory in its strictest sense, yet "this area lacks standardized reporting because metrics for forgetting and transfer gains vary significantly" [7]. *Tool/Gen* benchmarks (SWE-Bench, GAIA, ToolBench) emphasize process memory and traceability. Notably, no benchmark simultaneously evaluates all four desiderata: temporal consistency, multi-hop reasoning, collaborative access, and memory evolution.

3. Taxonomy of Graph-Based Agent Memory

From our classification of 30 graph-memory papers (2023–2026), 8 graph-focused MemAgents workshop papers, and production systems, we identify four established architectural patterns and two emerging ones (Table 4). Entity-relation KGs account for 60%; hierarchical event graphs for $\sim$7%; multi-graph composites, hypergraphs, and bipartite/associative graphs each appear in 1–3 papers. We note that this taxonomy describes *structural architecture*: real systems may span multiple patterns (e.g., GraphRAG uses entity-relation triples but clusters them hierarchically).

3.1. Pattern 1: Entity-Relation Knowledge Graphs

The classical KG represents memory as typed $(subject, relation, object)$ triples. CraniMem [21] combines a bounded FIFO episodic buffer with an entity-relation KG, using a neuroscience-inspired gating mechanism to filter irrelevant information before storage. A consolidation process transfers entries exceeding a replay threshold into the KG. F1 drops only 0.011 on HotpotQA under noisy input, versus 0.036 for Mem0.

UltRAG [22] queries *existing* large-scale KGs (116M entities, 1.6B relations) using ULTRA, a pretrained GNN foundation model. Personalized PageRank samples local subgraphs, and ULTRA executes structured queries achieving 92.7% on GTSQA—19–167$\times$ faster than baselines.

AriGraph [23] integrates episodic and semantic memory within a single KG: at each timestep, a new episodic vertex is added, then LLM-extracted triples update the semantic layer. MemLLM [24] integrates KG read/write directly into the LLM's forward pass. GraphRAG [18] clusters LLM-extracted entity-relation graphs via Leiden into hierarchical community summaries; LightRAG [19] reduces

overhead with incremental updates; PathRAG [20] retrieves relational paths for coherent reasoning.

Strengths and limitations. Entity-relation KGs excel at structured factual memory and multi-hop traversal. Their limitation is expressivity: a triple (s, r, o) captures only binary relations. A memory like "Alice, Bob, and Carol discussed the API migration at Tuesday's standup" involves a 4-ary relation that must be decomposed into multiple triples, losing the joint nature of the event.

3.2. Pattern 2: Hierarchical Event Graphs

These architectures maintain multiple graph layers—typically an episodic event layer and a semantic topic layer—with cross-layer edges enabling drill-down from abstractions to raw evidence.

GAM [25] maintains a composite memory graph $\mathcal{H}_t = \{G^{(t)}_{topic}, G^{(t)}_{event}, \mathcal{S}^{(t)}_{arch}, \mathcal{E}^{(t)}_{cross}\}$ consisting of a global *Topic Associative Network* G_{topic} (nodes = semantic clusters, edges = LLM-scored semantic correlations) and local *Event Progression Graphs* G_{event} (nodes = atomic utterances, edges = temporal/causal flow). A state-based consolidation mechanism implements a finite-state machine with two states: (1) an *Episodic Buffering State* that isolates transient context in the local G_{event}, protecting G_{topic} from noise, and (2) a *Semantic Consolidation State* triggered when an LLM-based semantic boundary detector identifies a topic shift ($b_t = 1$). Upon consolidation, the buffered subgraph is merged into a new topic node $v_{new} = \{c_{sum}, c_{raw}\}$ with dual-granularity representation (summary for thematic reasoning, raw content for fine-grained detail), integrated into G_{topic} via coarse-to-fine candidate selection (vector top-5 $\rightarrow$ LLM relation scoring with threshold τ).

Retrieval uses a three-stage *Graph-Guided Multi-Factor* process: (1) semantic anchoring in G_{topic} via vector similarity with 1-hop expansion, (2) structural drill-down via $\mathcal{E}_{cross}$ into archived event graphs, and (3) multi-factor re-ranking: $\text{Score}(v, q) = P_{sem}(v|q) \cdot \prod_k \beta_k^{\mathbb{I}_k(v,q)}$ with temporal (β_{time}=1.4), role-centric (β_{role}=1.4), and confidence (β_{conf}=1.2) boosting factors.

GAM reports Temporal F1 of 48.97 vs. 41.22 for Mem0 on Qwen 2.5-7B (+18.8%). However, a revealing exception emerges at GPT-4o-mini: Mem0 *outperforms* GAM on Temporal F1 (56.42 vs. 51.96). The authors attribute this to *temporal compression during consolidation*: the summarization step can lose fine-grained chronological cues that raw retrieval preserves—a fundamental tension in hierarchical event graphs. Notably, GAM's temporal factor (β_{time}=1.4) is a *binary* boost for temporal relevance (not continuous recency decay): it activates when a memory contains temporal expressions matching a time-sensitive query.

The ablation reveals that the Event Progression Graph is the most critical component (-37.4% F1 when removed), followed by the state-switching mechanism (-18.6%)—confirming that the hierarchical separation between buffering and consolidation, not just graph structure, drives performance. GAM also dominates AriGraph across all backbones (Avg F1: 40.00 vs. 19.34 at 7B), with the largest gap in temporal reasoning (48.97 vs. 5.51).

G-Memory [26] extends to multi-agent systems with three tiers: an insight graph, a query graph, and an interaction graph. MemFly [27] implements a three-layer hierarchy (Note–Keyword–Topic) grounded in the Information Bottleneck principle, with Leiden-based topic clustering and tri-pathway retrieval fused via Reciprocal Rank Fusion (F1 44.4% on LoCoMo with GPT-4o). The reconstructive graph memory of [28] introduces a Cue-Tag-Content heterogeneous graph where retrieval is active reconstruction: the LLM iteratively selects which graph paths to follow, provably more powerful than passive retrieval.

Talebirad et al. [140] provide a unifying theoretical framework for all hierarchical memory systems via three formal operators: *extraction* (α) maps raw data to atomic information units, *coarsening* ($C = (\pi, \rho)$) partitions units into groups and assigns a representative to each, and *traversal* (τ) selects which units to surface given a query and token budget. The central theoretical insight is a *self-sufficiency spectrum* for the representative function ρ: when self-sufficiency $\text{SS}(\rho, G_j) = I(G_j; \rho(G_j))/H(G_j)$ is high (e.g., LLM-generated summaries in RAPTOR), collapsed search over representatives suffices; when low (e.g., category labels in xMemory), top-down refinement is required. This *coarsening-traversal coupling* constrains viable retrieval strategies and explains why mismatched ρ and τ waste tokens. Instantiated

across eleven systems (including RAPTOR, GraphRAG, SimpleMem, GAM, and MemBrain), the (α, C, τ) decomposition provides the first formal comparison language for hierarchical memory architectures.

Strengths and limitations. Hierarchical event graphs naturally capture episodic-to-semantic consolidation. Their limitation is temporal grounding: none implements validity intervals on semantic nodes or tracks when abstracted knowledge becomes stale.

3.3. Pattern 3: Bipartite and Associative Graphs

These lightweight architectures use two-type graph structures without typed relations, where traversal is often approximated via vector operations.

LP-RAG [31] constructs a bipartite chunk-query graph with mutual k-NN edges. Retrieval is cast as link prediction: a GNN (NCN architecture) predicts chunk-query links, outperforming embedding similarity by 1.6–2.8% EM on multi-hop QA. EcphoryRAG [29] builds a bipartite entity-chunk graph with centroid-based spreading activation, achieving 3.3× reduction in indexing tokens versus HippoRAG while maintaining competitive multi-hop performance. EMG-RAG [30] uses RL-based optimization over editable memory graphs for personalized agents. AssoMem [99] constructs an associative memory graph that anchors dialogue utterances to automatically extracted clues (entities, attributes, temporal cues), enabling multi-signal retrieval that exploits associative pathways rather than relying solely on semantic distance—particularly effective in similarity-dense scenarios where standard embedding retrieval returns redundant context.

Strengths and limitations. Bipartite graphs minimize construction overhead and enable fast updates. Without typed relations or hierarchical layers, they cannot represent complex knowledge structures or support causal reasoning.

3.4. Pattern 4: Multi-Graph Composites

The most recent direction maintains multiple *orthogonal* graphs, each capturing a different memory dimension.

MAGMA [32] represents memory over *unified event-nodes* connected by four orthogonal edge types: temporal (chronological ordering), causal (cause-effect dependencies), semantic (topical similarity), and entity (shared participant). A dual-stream memory architecture separates fast synaptic ingestion (rapid event capture) from slow structural consolidation (periodic reorganization). Retrieval uses intent-aware adaptive traversal via heuristic beam search with RRF-based (Reciprocal Rank Fusion) anchor identification: the query intent determines which graph views to prioritize. On LoCoMo, MAGMA achieves a Judge score of 0.700 (best) at 1.47 s latency (lowest among graph methods); ablation confirms all four edge types are necessary—removing any one degrades performance. TM2RAG [33] extends this approach with five specialized graphs—entity (directed), causal (DAG), temporal (tree: Year→Month→Event), semantic (directed), and persona (directed)—unified through a meta graph with namespaced IDs and cross-graph edges. An agentic pipeline classifies query intent and routes to the appropriate graph combination. AMA-Bench [34] introduces a Causality Graph extracting inter-state causal dependencies from agent trajectories; removing it drops performance by 24%.

Strengths and limitations. Multi-graph composites are the most expressive architecture but introduce coordination overhead: how should updates propagate across graphs, and how should conflicting information be resolved?

3.5. Emerging Patterns

Hypergraph memory. HyperGraphRAG [35] uses hyperedges to represent n-ary relational facts directly. HyperMem [36] implements a three-level hypergraph (topics, episodes, facts). HGMem [37]

groups multi-step reasoning chains via hyperedges. All treat the hypergraph as a *data structure*—none applies hypergraph neural networks or higher-order message passing (§8).

Procedural strategy graphs. The trainable graph memory of [39] represents agent trajectories as a heterogeneous directed acyclic graph $\mathcal{G} = (V, E, \mathcal{O}_V, \mathcal{R}_E, C)$ with three node types and learnable edge weights:

1. **Query layer** ($\mathcal{Q}$): Each node q_i stores a task instance with input, execution trajectory, and outcome label.
2. **Transition path layer** ($\mathcal{T}$): Each node t_j encodes a canonical decision pathway derived from a Finite State Machine abstraction that maps raw execution traces to cognitive states (e.g., `StrategyPlanning`, `ToolExecution`, `KnowledgeUncertainGap`).
3. **Meta-cognition layer** ($\mathcal{M}$): Each node m_k encodes a high-level strategic principle distilled from contrasting successful and failed paths—generalizable heuristics for problem-solving.

Edges $\mathcal{Q} \to \mathcal{T} \to \mathcal{M}$ form a directed acyclic topology with learnable weights w_{qt}, w_{tm}. A REINFORCE-based optimization procedure calibrates these weights via counterfactual reward gaps: $\Delta R_k = R_{with}(m_k) - R_{w/o}$, strengthening paths to meta-cognitions that improve downstream task performance. At inference, a relevance score $\rho(m_k|q_{new}) = \sum \text{Sim}(q_{new}, q_i) \cdot w_{qt}^{(i,j)} \cdot w_{tm}^{(j,k)}$ selects the top-k meta-cognitions as strategic priors.

PRAXIS [97] implements a complementary procedural memory for web agents, where each memory entry stores a full state-action tuple $(M_i^{env\text{-}pre}, M_i^{int}, a_i, M_i^{env\text{-}post})$—environment state before and after the action, plus the agent's internal state. Retrieval jointly matches environmental state (via IoU) and internal state (via embedding similarity), grounded in the psychological principle of *state-dependent memory* [97]. On the REAL web benchmark, procedural memory improves accuracy from 40.3% to 44.1% (+9.4%), reliability from 74.5% to 79.0%, and reduces steps-to-completion from 25.2 to 20.2 across five VLM backbones —demonstrating that state-indexed action exemplars provide reusable procedural priors that generalize across tasks. LEGOMem [100] takes a modular approach to procedural memory, decomposing workflows into composable memory modules for multi-agent systems—enabling agents to share and reuse procedural knowledge across different workflow automation tasks. SkillRL [109] closes the loop between experience and policy by building a hierarchical SkillBank via experience-based distillation: successful trajectories become demonstrations, failed ones become concise failure lessons. The skill library is organized into *general* skills $\mathcal{S}_g$ (universal strategies) and *task-specific* skills $\mathcal{S}_k$ per category. Crucially, the skill library *co-evolves* with the agent's policy during RL training via GRPO: after each validation epoch, failed categories trigger skill evolution $\mathcal{S}_{\text{new}} = \mathcal{M}_T(\mathcal{T}_{\text{val}}^-, \text{SkillBank})$. On ALFWorld and WebShop, SkillRL achieves state-of-the-art with 14% improvements, and skill distillation achieves 10–20$\times$ token compression versus raw trajectory storage.

Proced-Mem [112] provides the first benchmark isolating *procedural memory retrieval* from downstream execution, across two domains: ALFWorld (336 trajectories, 6 retrieval methods) and OSWorld (206 computer tasks, 7 methods spanning text, visual, and multimodal). Two key findings emerge: (1) a *generalization cliff* where embedding methods suffer 30–42% MAP degradation on out-of-distribution queries while LLM-generated abstractions degrade by only 11%, and (2) a *granularity-method reversal* where visual embeddings rank first at coarse pattern matching but last at fine-grained procedural clustering—a "visual similarity trap" where screenshots capture application identity rather than procedural structure.

Uniquely, the trainable graph memory integrates memory into the RL *training* loop: meta-cognitive strategies are injected as context priors during policy optimization via GRPO, not merely at inference. The trained Qwen3-4B model (0.426 EM) outperforms the baseline Qwen3-8B (0.395), demonstrating that procedural graph memory can substitute for model scale (§5.7). Remarkably, the entire memory graph is constructed from only 1,000 HotpotQA examples yet generalizes across 6 out-of-domain QA datasets; even a single meta-cognition (k=1) yields +23.3% improvement. A *speculative induction* mechanism addresses cold start: when a new query produces only failures, the system retrieves semantically similar

queries with successful paths and borrows their meta-cognitions as speculative guidance. The memory is also model-agnostic: graphs built with GPT-4o and Gemini-2.5-pro yield equivalent downstream performance.

3.6. Graph Construction Pipelines

How memory graphs are built from raw experience determines the quality ceiling for all downstream operations. The survey [7] identifies three extraction modalities:

From text. The dominant pipeline uses LLM-based triple extraction: the LLM parses observations into $(head, relation, tail)$ triplets, followed by an integration phase with conflict detection and relationship pruning. Mem0 [1] evaluates new memories against existing ones via semantic gating; AriGraph [23] incrementally updates a shared episodic-semantic KG at each timestep. LLM-based extraction outperforms traditional NER/RE because LLMs "generalize to novel fine-grained entity types and exploit implicit relations based on contextual understanding" [7].

From trajectories. Sequential agent traces require event segmentation and timestamping (Reflexion), dynamic state snapshots at key decision points (Mem-α [67]), or offline pattern mining to discover frequent subsequences abstracted as procedural memory templates.

From multimodal data. Vision-language models generate textual descriptions, object detectors produce scene graphs, and CLIP-style encoders create joint multimodal embeddings. Optimus-1 [91] combines structured perceptual extraction with hierarchical KG construction for embodied agents. MemoryVLA [118] bridges this to robotic manipulation with a dual-memory system inspired by cognitive science: a working memory buffer for short-lived sensory representations (analogous to prefrontal cortex) and a hippocampal-inspired episodic/semantic long-term store that preserves both verbatim task details and gist summaries. M3-Agent [119] processes real-time visual and auditory inputs into entity-centric multimodal memories, enabling multi-turn reasoning over accumulated world knowledge.

3.7. Summary

Table 4 and Figure 2 compare the four established patterns across seven capability dimensions. The radar chart makes the trade-off structure visually explicit: entity-relation KGs offer balanced coverage, hierarchical event graphs excel at consolidation and temporal ordering, and multi-graph composites provide the broadest capability profile at the cost of scalability. Production deployments increasingly adopt multi-strategy retrieval combining vector search with graph traversal and temporal filtering [16, 6], suggesting that the highest-performing systems will span multiple patterns.

Table 3 provides a comprehensive comparison of key systems across architectural dimensions.

Comparison with survey taxonomies. The comprehensive survey by Yang et al. [7] proposes a complementary taxonomy organized by memory lifecycle (extraction, storage, retrieval, evolution) with 5 graph storage types, 9 retrieval operators, and 2 evolution paradigms. Their taxonomy is broader on retrieval (6 base operators + 3 enhancement strategies vs. our 5-level spectrum) and deeper on extraction (3 data modalities with 9 sub-techniques). Our taxonomy adds distinctions they do not make: bipartite associative graphs as a separate pattern (folded into their hypergraph category), procedural strategy graphs (our category for TGM, not identified in the survey), and causal graphs distinct from KGs. A key structural difference: they treat temporal graphs as a first-class storage type alongside KGs and hierarchical structures, while we treat temporal mechanisms as an orthogonal dimension applicable to any graph type (§4). Our retrieval spectrum is organized by *learning intensity* (graph algorithms → RL → GNN → TDL); theirs by *operator type* (similarity, rule, temporal, graph, RL, agent)—complementary perspectives that together provide a complete picture. The survey extends CoALA [11]

Table 3
Comprehensive comparison of graph-based agent memory systems. Temporal: ✓ = deep mechanism, △ = timestamps/decay, — = none. Retrieval levels: GA = graph algorithms, MS = multi-strategy, RL = reinforcement learning, GNN = message passing, AB = agent-based, PR = post-retrieval.

System	Graph Type	Temp.	Retr.	Evolution	Best Score	Benchmark
Production & multi-strategy systems						
Zep/Graphiti	KG (bi-temporal)	✓	MS	Temporal invalidation	94.8%	Deep Mem. Retrieval
Hindsight	KG + vector	△	MS	—	91.4%	LongMemEval
FalkorDB+Mem0	KG (graph DB)	—	MS	Dedup/merge	<140 ms p99	Production
Hierarchical & multi-graph architectures						
GAM	Hierarchical (2-layer)	△	GA	State-based consol.	40.00 F1	LoCoMo (7B)
MAGMA	Multi-graph (4 types)	✓	GA	Dual-stream consol.	0.700	LoCoMo Judge
MemFly	Hierarchical (3-layer)	—	GA	IB clustering	44.4 F1	LoCoMo (GPT-4o)
H-MEM	Hierarchical (GNN)	—	GNN	Index routing	—	Long-term reason.
Entity-relation KG systems						
Mem0	KG (entity-relation)	—	GA	Dedup/merge	28.86 F1	LoCoMo (7B)
HippoRAG	KG (open)	—	GA	—	+3–10 F1	Multi-hop QA
CraniMem	KG (gated buffer)	—	GA	Replay consol.	−0.011 F1	HotpotQA noise
Temporal-aware systems						
RecallM	KG (temporal window)	✓	GA	Context revision	4× vs Vec	Temporal QA
MemoTime	KG (Tree of Time)	✓	GA	Temporal monoton.	—	Temporal reason.
TReMu	KG (neuro-symbolic)	✓	GA	Code-gen dates	—	Multi-session
Mnemosyne	KG (edge-based)	△	GA	Recency decay	—	Long-term mem.
LiCoMemory	KG (lightweight)	△	GA	Decay + hyperlinks	—	Long-term reason.
RL & learned retrieval systems						
TGM	Procedural (3-layer)	—	RL	REINFORCE weights	0.426 EM	HotpotQA (4B)
Memory-R1	KG	—	RL	RL operations	—	Memory via RL
MAQR	KG	—	RL	Q-value select.	—	Query reconstr.
Mem-α	KG (RL-built)	—	RL	State snapshots	—	RL benchmarks
GNN & higher-order systems						
LP-RAG	Bipartite	—	GNN	—	+2.8% EM	Multi-hop QA
MemoGraph	KG + RGCN	—	GNN	—	86.9%	MATH (7B)
HyperMem	Hypergraph (3-level)	—	GA	—	—	—
Multi-agent & collaborative systems						
Collabor Mem.	KG (shared)	—	AB	Access control	—	Multi-agent
DAMCS	Hierarchical (decentr.)	—	AB	Adaptive merge	—	Cooperative plan.
MIRIX	KG (shared)	—	AB	Inter-agent sharing	—	Multi-agent
GraphCogent	KG (distributed)	—	AB	Collab. graph	—	Working mem.
Procedural & domain-specific systems						
PRAXIS	State-action tuples	—	GA	Real-time learn.	44.1%	REAL (web)
LEGOMem	Modular procedures	—	GA	Workflow reuse	—	Workflow auto.
AssoMem	Assoc. graph	—	GA	Clue anchoring	—	Memory QA
Entropic Mem.	Two-tier (hot/cold)	—	GA	Free-energy consol.	0.29 SR	Infinite Room
TierMem	Two-tier (provenance)	—	MS	Verified write-back	0.851 acc	LoCoMo
FactorMiner	Skill + experience	—	GA	Ralph Loop	—	Alpha mining
A-MAC	Admission gate	—	MS	5-signal scoring	0.583 F1	LoCoMo
SkillRL	Hier. SkillBank	—	RL	Recursive evol.	+14%	ALFWorld
SimpleMem	Multi-view index	—	MS	Sem. compression	+26.4% F1	LoCoMo
JitRL	(s, a, G_t) triplets	—	—	Test-time optim.	SOTA	WebArena

with 6 cognitive memory types (adding associative, working, and sentiment memory) and introduces a knowledge-vs.-experience axis we do not explicitly cover.

4. Temporal Knowledge Graphs for Agent Memory

The central promise of graph-based memory is temporal grounding: tracking when knowledge was true, when it was learned, and when it became stale. Yet 57% of graph-memory papers implement no temporal mechanism, and only 20% go beyond timestamps to temporal KG embeddings or validity intervals.

Table 4
Four established architectural patterns for graph-based agent memory.

	Bipartite Assoc.	Entity-Rel. KG	Hierarchical Event	Multi-Graph Composite
Multi-hop traversal	△	✓	✓	✓
Temporal ordering	—	△	✓	✓
Validity intervals	—	△	—	△
Causal structure	—	—	—	✓
Consolidation	—	—	✓	△
Incremental update	✓	✓	✓	△
Scalability	✓	✓	△	—

✓ = supported, △ = partial, — = absent. Columns ordered by increasing structural complexity.

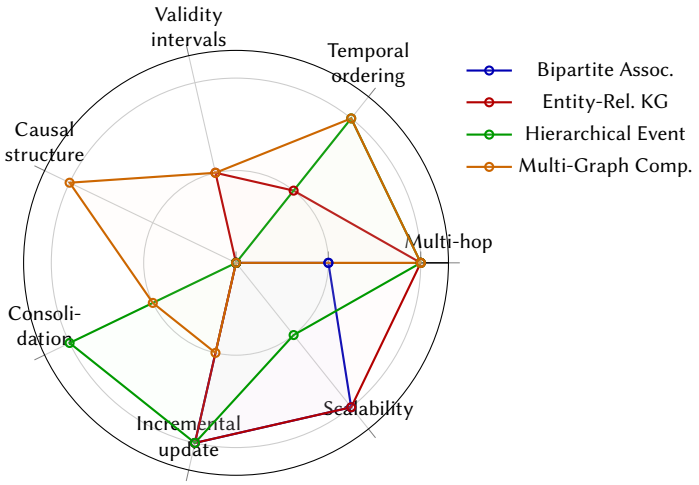

Figure 2: Radar chart comparing the four established architectural patterns. Values: 1.0 = supported (✓), 0.5 = partial (△), 0 = absent (—). Multi-graph composites cover the most dimensions but sacrifice scalability; entity-relation KGs provide balanced coverage; bipartite graphs trade expressivity for efficiency.

4.1. The Bi-Temporal Model

Graphiti/Zep [14] provides the most complete temporal solution through a *bi-temporal* model distinguishing *event time* (when something happened) from *ingestion time* (when the system learned it). Every edge carries validity intervals $(t_{valid}, t_{invalid})$; old facts are invalidated, not deleted. Queries can be time-scoped: "what was true about X at time T?" This achieves 18.5% accuracy improvement on LongMemEval temporal tasks.

In the broader corpus, only 5 of 30 papers implement temporal KG mechanisms: Zep, MAGMA [32], MemoTime [56], TReMu [57], and a TKG approach for partially observable environments. No workshop paper implements bi-temporal tracking.

RecallM [60] implements a different temporal model using *temporal windows* rather than validity intervals. A global temporal index t is incremented on each knowledge update; every concept N_i and relation E_i maintains a temporal index $T(x)$ updated when touched. Retrieval applies a window constraint: only concepts satisfying $T(N_i) - s \leq T(E_i) \leq T(c_i)$ are considered, where s is the window size. Combined with Hebbian-inspired relation strength (co-occurrence increments by 1) and a composite ordering $s(r) + \alpha t(r)$ (α=3), this achieves implicit temporal validity without explicit intervals. RecallM is 4× more effective than vector databases for temporal belief updating, maintaining $\sim$90% accuracy after 25 knowledge update cycles where VectorDB drops to $\sim$20%.

4.2. Temporal KG Embeddings

The temporal KG literature offers learned representations not yet applied to agent memory. TNTComplEx extends ComplEx with a temporal factor; TeAST maps temporal evolution onto Archimedean spiral geometry; recent work uses Lie Group manifolds [44]. Temporal Graph Networks (TGN) [45] maintain per-node memory states—conceptually analogous to per-entity agent memory. EvoReasoner [46] demonstrates temporal multi-hop reasoning over evolving KGs with 18.4% accuracy gains.

MemoTime [56] takes a structural approach, organizing temporal reasoning into a hierarchical "Tree of Time" that enforces *temporal monotonicity*: any retrieved reasoning path $e_1 \to e_2 \to e_3$ strictly adheres to $t_1 \leq t_2 \leq t_3$. TReMu [57] addresses temporal ambiguity in multi-session dialogues (e.g., "last Friday") by decoupling *mention time* (session timestamp) from *event time* (inferred absolute date), generating Python code for precise date arithmetic before retrieval. These represent the frontier of temporal reasoning in agent memory, yet most methods await integration into production systems.

4.3. Neuroscience-Inspired Temporal Mechanisms

Several systems draw explicit inspiration from cognitive science. Neural Graph Memory [74] implements Hebbian-style associative retrieval: frequently co-retrieved memories have their connecting edge weights strengthened, mimicking long-term potentiation in biological neural networks. This creates emergent temporal structure without explicit timestamps—recently and frequently accessed memories naturally surface. Mnemosyne [75] implements three neuroscience principles: (1) Ebbinghaus forgetting where memory strength decays exponentially unless reinforced, (2) spreading activation where querying one concept propagates activation through graph edges, and (3) spaced repetition via a "rewind" mechanism that boosts strength upon re-access. On LoCoMo temporal reasoning, Mnemosyne achieves 60.42%—notably strong compared to GAM's 48.97% at 7B, though evaluated with GPT-4. MemoryBank [76] implements an Ebbinghaus forgetting curve, where memory retention follows $R(t) = e^{-t/S}$ with strength S increasing upon each retrieval —significance-based pruning removes rarely accessed nodes, preventing unbounded growth while preserving high-utility memories. LiCoMemory [77] combines a lightweight decay function that reduces weights of older facts with explicit hyperlinks from summary nodes to evidence chunks, enabling efficient hierarchical retrieval with temporal awareness at minimal computational cost. Park et al. [129] take inspiration from human cue-dependent recall: rather than retrieving by embedding similarity, the agent autonomously generates recall cues from the conversational context, coupled with a mathematical model for dynamic memory consolidation that quantifies how multiple memories compete for retrieval—addressing a fundamental limitation in temporal cognition that static embedding-based retrieval cannot capture.

ReSuME [139] proposes a fundamentally different write-gate signal: instead of external heuristics or embedding similarity, memory formation is triggered by *representational surprise*—compression failure in the model's own latent space. A Sparse Autoencoder (SAE) trained on routine conversational activations approximates the manifold of typical internal states; the reconstruction error $\epsilon_t = \|x_t - \hat{x}_t\|_2$ serves as an unsupervised, model-internal signal for memory importance. Memory writes occur only when a calibrated surprise score exceeds a threshold ($s_t > \tau$), transforming episodic gating from an externally imposed rule to a geometric decision grounded in the model's representational dynamics. Under fixed memory budgets, surprise-gated writing achieves a superior performance–memory trade-off compared to heuristic and similarity-based baselines, with covariance-aware normalization enabling robust cross-domain calibration—suggesting that episodic memory in neural agents need not be imposed through external rules but can emerge from intrinsic latent geometry.

Entropic Memory [93] draws from thermodynamic annealing rather than neuroscience, formalizing memory consolidation as free-energy minimization: $F = E + \lambda S$, where $E = -\text{Utility}(m)$ (query relevance and recency) and $S = H(\mathbf{e}_m)$ is the Shannon entropy of the memory's embedding distribution. A two-tier architecture (hot working buffer M_{hot} and cold long-term store M_{cold}) periodically consolidates via a stochastic acceptance rule $P = \min(1, \exp(-\Delta F/T))$, where temperature T controls the exploration-exploitation trade-off. The entropy term penalizes noisy or ambiguous memories even

when frequently accessed—at 50% noise, Entropic Memory maintains survival rate $SR \approx 0.28$ while greedy importance sampling degrades to 0.24 (+15% relative), demonstrating robustness to distractors that utility-only heuristics cannot filter.

MIRROR [136] validates the CLS prediction that fast encoding and slow consolidation serve *complementary* functions: an Inner Monologue Manager rapidly encodes turn-specific experience into parallel threads (Goals, Reasoning, Memory), while a Cognitive Controller performs $O(1)$ reconstructive consolidation—regenerating a bounded first-person narrative each turn rather than accumulating $O(n)$ traces. On CuRaTe, MIRROR achieves 21% relative improvement across seven architectures; ablation confirms the integrated system outperforms either component alone (+1–8% synergy). Lerma-Torres [137] maps twelve principles from neuroscience and clinical psychology to computational mechanisms: complementary learning systems (dual WM+KG store), emotional valuation via somatic-marker-inspired valence vectors, CBT belief hierarchies as emergent weight distributions, System 1/2 retrieval routing, conviction tracking with precision scalars, and thalamic gating for evidence routing—proposing seven functional properties any neuroscience-grounded memory must satisfy.

Belief Engine [116] addresses a related but distinct problem: LLM agents can retrieve memories but cannot *maintain stable beliefs* across interactions, exhibiting random drift or pathological inertia. By externalizing belief state into a Bayesian log-odds update rule with tunable parameters for evidence sensitivity (K), prior adherence (K_a), and asymmetric weighting (B), Belief Engine produces stance trajectories that are smoother and more reproducible than standard agentic updating. The separation of fast generative reasoning (LLM) from slower reflective updating (Belief Engine) mirrors dual-process theories from cognitive science.

Baldelli et al. [152] prove a fundamental limitation: standard chat-based agents (Public-Only Chat Agents, POCAs) are *structurally incapable* of maintaining private state across turns. Their impossibility theorem shows that for Private State Interactive Tasks (PSITs)—games, negotiations, medical simulations requiring hidden information—no agent restricted to public conversation history can simultaneously preserve secrecy and consistency. All major commercial LLMs (ChatGPT, Gemini, Claude) fail empirically. The solution requires explicit *private working memory*: a separate mutable store updated each turn and reinjected into the agent's context, establishing private state maintenance as a necessary memory component distinct from the episodic/semantic/procedural triad. MARTA [153] proposes *thermodynamic memory regulation*: rather than the "Retrieve-Always" paradigm where every query triggers external memory access, MARTA measures the agent's internal predictive entropy and token margin to determine whether retrieval is epistemically necessary. By projecting these uncertainty signals onto a learned *Affordance Manifold*, MARTA arbitrates between parametric knowledge (System 1) and non-parametric memory (System 2) at the pre-softmax level—before the first token is even generated. This achieves 3.7× throughput improvement with no token overhead, matching active RAG fidelity while eliminating wasteful retrievals.

These biologically and physically grounded mechanisms represent a complementary approach to the formal temporal models of Zep and MemoTime: rather than explicit validity intervals, they achieve implicit temporal currency through access-pattern dynamics or energy-based consolidation criteria.

4.4. The Staleness Gap

Missing temporal mechanisms cause *knowledge staleness*. In SE, LLMs generate deprecated API calls 25–38% of the time [47]; LoCoMo-Plus shows all systems converge to ∼20% accuracy at 3+ months temporal distance. The mechanism is *semantic drift*: later queries use different language than original cues. Without temporal structure, similarity-based retrieval breaks down; graph structure with validity intervals enables time-scoped retrieval robust to this drift.

5. From Graph Algorithms to Learned Retrieval

Memory retrieval over graphs spans a spectrum from training-free algorithms to fully learned neural approaches (Figure 3). Agent memory graphs are built incrementally, unique per user, and change

Graph Algorithms	Multi-Strategy Heuristics	RL / Differentiable	GNN Message Passing	Geometric DL
PPR, BFS, Leiden, recency decay	Semantic + BM25 + graph + temporal	Gumbel-Softmax, RL edge optim.	NCN, RGCN, VGAE link prediction	Hyperbolic, curvature, Lie groups, ODEs
HippoRAG, EcphoryRAG, GraphRAG, GAM	Hindsight, Cognee, FalkorDB+Mem0	D-RAG, Trainable Graph Mem., EMG-RAG	LP-RAG, MemoGraph, GNN-RAG	(no agent memory system yet)
0 training	Pre-trained ranker only	Task-reward training	Graph-structure training	Geometric training
10–150 ms	100–600 ms	Varies	Varies	Unknown

Figure 3: The retrieval spectrum for graph-based agent memory. The field is concentrated at the left: graph algorithms and multi-strategy heuristics dominate. RL-based approaches now span five distinct formulations. Only four systems apply GNN message passing. Post-retrieval verification (§5.5) provides an orthogonal enhancement. Geometric deep learning remains an unexplored frontier (§8).

every session—properties that favor training-free methods. Yet the cognitive ceiling (below 25% on LoCoMo-Plus) may require learned representations.

5.1. Training-Free Graph Algorithms

The dominant approach requires no training. HippoRAG [9] implements Personalized PageRank as a neurobiologically inspired pattern-completion mechanism. EcphoryRAG [29] approximates spreading activation via centroid-based expansion in embedding space. GraphRAG [18] and MemFly [27] use Leiden for unsupervised community detection. GAM [25] applies exponential recency decay. These algorithms work on any graph at 10–150 ms, explaining why every deployed system adopts them.

5.2. Multi-Strategy Heuristic Retrieval

Production systems combine multiple training-free pathways. Hindsight [16] executes semantic search, BM25, graph traversal, and temporal filtering simultaneously, fusing via a pre-trained cross-encoder (91.4% on LongMemEval at 100–600 ms). FalkorDB+Mem0 [15] and Cognee similarly combine graph traversal with embedding search. The only learned component is a pre-trained re-ranker—no per-agent training required. SimpleMem [110] achieves a new Pareto optimum on the performance-efficiency frontier via three-stage semantic lossless compression: (1) *Semantic Structured Compression* uses LLM attention as a density gate $\Phi_{\text{gate}}(W) \to \{m_k\}$ to filter low-entropy dialogue into context-independent memory units with coreference resolution and temporal anchoring; (2) *Online Semantic Synthesis* consolidates related fragments on-the-fly during the write phase; (3) *Intent-Aware Retrieval Planning* decomposes queries into parallel semantic, lexical (BM25), and symbolic paths with adaptive retrieval depth d. On LoCoMo, SimpleMem achieves 26.4% F1 improvement over Mem0 while reducing inference-time tokens by 30×—occupying the top-left corner of the performance-vs.-cost space. Store routing complements multi-strategy retrieval at a higher level of abstraction: rather than choosing retrieval *methods*, the agent must choose which memory *stores* (STM, summaries, LTM, episodic) to query. Gaikwad [117] formalizes this as a cost-sensitive subset-selection problem $\pi^*(q) = \arg\max_{G \subseteq \mathcal{S}} \left[\mathbb{E}[\text{Acc}(q, G)] - \lambda \sum_{s \in G} c_s\right]$, showing that an oracle router achieves higher QA accuracy with substantially fewer tokens than uniform retrieval—demonstrating that selective store access both reduces cost and improves signal-to-noise. General Agentic Memory (GAM-JIT) [161] challenges the dominant *ahead-of-time* (AOT) paradigm where memory is pre-compressed offline and retrieved at query time. Observing that *memorization is a form of data compression and thus inevitably subject to information loss*, GAM-JIT follows a *just-in-time* (JIT) compilation principle: a **Memorizer** maintains both a lightweight summary and a complete page-store of decorated session histories, while a **Researcher** performs iterative deep research at query time via a Plan→Search→Reflect loop using embedding, BM25, and ID-based retrievers over the page-store. The reflection step determines whether gathered information fully satisfies the request; if not, a new

research round is triggered. Both modules are jointly optimized via REINFORCE. GAM-JIT outperforms all baselines across four benchmarks (LoCoMo, HotpotQA, RULER, NarrativeQA), achieving 90.2% accuracy on RULER at 448K tokens (vs. 80.0% for the best baseline) and 97.7 F1 on HotpotQA-56K. Critically, GAM-JIT maintains strong performance even when the memorizer backbone is downsized to Qwen-2.5-0.5B, demonstrating that the researcher's retrieval quality matters more than compression quality—a finding consistent with the diagnostic result that retrieval method dominates over write strategy [154].

5.3. RL and Differentiable Optimization

A middle ground applies learning to memory *management* rather than graph structure. D-RAG [55] enables joint retriever-generator optimization through Gumbel-Softmax subgraph sampling. The trainable graph memory of [39] applies REINFORCE to optimize edge weights in a heterogeneous memory graph, achieving +25.8% improvement at 4B scale (§5.7). EMG-RAG [30] uses RL to learn when to modify edges.

Several systems explore distinct RL formulations for the same underlying problem. Memory-R1 [61] treats memory *operations* (add, delete, retrieve) as learnable actions within a PPO/GRPO framework—the agent autonomously learns an optimal policy for memory management itself. MAQR [66] introduces Q-value-based action selection for memory-augmented query reconstruction: the retrieval agent learns an action-value function $Q(s, a)$ where the state s encodes the current query and agent context, and actions a correspond to graph navigation steps, enabling principled exploration-exploitation trade-offs in retrieval. Mem-α [67] takes a different approach, constructing the memory graph itself via reinforcement learning: dynamic state snapshots capture agent-environment states at key decision points, and the RL policy determines which snapshots to retain, merge, or discard. Reflective Memory [68] combines prospective and retrospective reasoning for long-term personalized dialogue, using retrospective temporal reasoning to reinterpret past memories in light of new information. MemSearcher [69] trains an end-to-end RL policy that jointly learns to reason, search, and manage memory, unifying the typically separate retrieval and reasoning stages.

These systems collectively demonstrate that RL for graph memory spans a spectrum from learning *edge weights* (TGM) to learning *memory operations* (Memory-R1) to learning *graph construction* (Mem-α) to learning *unified retrieval-reasoning* (MemSearcher). All learn *policies over graphs* but not *representations from graph structure*—the training signal comes from task reward, not graph topology.

5.4. GNN Message Passing

Only a handful of systems apply actual GNN message passing to agent memory. LP-RAG [31] recasts retrieval as inductive link prediction on a chunk-query bipartite graph. Four architectures were evaluated (NCN, VGAE, SEAL, PEG); NCN outperformed embedding similarity by 1.6–2.8% EM on multi-hop QA, generalizing inductively to new queries. MemoGraph [40] encodes an evolving reasoning graph via RGCN with attention-based readout, achieving 86.9% on MATH with a 7B model.

H-MEM [70] implements hierarchical memory with index-based routing layer by layer, using GNN cross-layer representations to score and propagate information between abstraction levels. Unlike LP-RAG's flat bipartite structure, H-MEM's GNN operates over a hierarchical DAG, enabling multi-resolution retrieval where coarse-grained topic nodes guide traversal to fine-grained evidence. CAN [71] directly addresses the question "Can graph learning improve planning in LLM-based agents?" by applying GNN-based cross-layer representations for planning tasks, demonstrating that learned graph structure can improve agent decision-making beyond retrieval alone.

KG-Agent [125] demonstrates that even a small LLM can autonomously reason over KGs through a multifunctional toolbox and KG-based executor, with a knowledge memory that accumulates discovered facts during multi-hop traversal—showing that the graph itself can serve as both memory and reasoning substrate. RRP (Reliable Reasoning Path) [160] addresses a complementary challenge: generating *structurally consistent* reasoning paths from KGs rather than merely retrieving isolated

triples. RRP combines semantic reasoning (LLM-generated candidate paths minimizing KL divergence) with structural reasoning (relation embeddings via bidirectional distribution learning), then applies a *rethinking module* that ranks paths by dual semantic-structural similarity to eliminate redundant and conflicting chains. On WebQSP, RRP achieves 90.0% Hits@1 (vs. 85.7% for RoG) with only a 7B backbone and no fine-tuning—demonstrating that organizing KG relations into coherent reasoning paths, rather than feeding raw triples, substantially improves multi-hop accuracy.

GNN-RAG [10] and UltRAG [22] apply GNNs to external KGs (not agent memory), but demonstrate that GNN-based reasoning can transfer zero-shot across graphs—a property critical for dynamic memory. UltRAG's ULTRA foundation model, pre-trained on diverse KGs, achieves 92.7% on structured QA while being 19–167$\times$ faster than baselines, suggesting a path toward pre-trained GNN models that generalize across memory graphs.

5.5. Post-Retrieval Verification and Enhancement

A distinct class of systems applies graph-based verification *after* initial retrieval. SimGRAG [72] retrieves candidate subgraphs via standard methods, then verifies relevance by *imagining* a query-specific subgraph and matching it against retrieved candidates via graph alignment. This post-retrieval simulation catches false positives that embedding similarity misses, particularly for multi-hop queries where individual nodes appear relevant but the relational structure does not match. MemGen [73] generates latent memory token representations as an intermediate step, effectively creating a compressed query-specific memory representation before retrieval. The survey [7] identifies post-retrieval enhancement as an underexplored strategy, with only 3 of 78 systems implementing it—yet it addresses a key failure mode of graph-based retrieval: structurally relevant subgraphs may contain semantically irrelevant nodes when traversal heuristics over-expand.

5.6. Why So Few Learned Approaches?

The scarcity (4/78 systems) reflects a fundamental tension:

1. **Cold start**: An agent's memory begins empty. There is no graph to train on until training-free algorithms have already been serving retrieval.
2. **No retrieval labels**: GNN training requires supervision (which nodes are relevant for which queries?) that varies per task. LP-RAG trains on synthetic pairs; general solutions remain elusive.
3. **Graph evolution outpaces retraining**: The memory graph changes every session. Inductive GNNs partially address this, but retraining costs accumulate.
4. **Diminishing returns on factual tasks**: Heuristic multi-strategy retrieval already achieves 91.4% on LongMemEval; LP-RAG's GNN improves by only 1.6–2.8%.

Houliston et al. [134] provide a theoretical foundation for *why* external memory dominates: in-weight memorization requires parameters growing linearly with the number of facts regardless of data compressibility, while tool-augmented retrieval from external stores (including graph memory) can handle unbounded facts via a simple, efficient circuit construction. Empirically, teaching LLMs tool use is significantly more effective than fine-tuning new facts into weights—a formal argument for graph-based external memory over parametric knowledge.

ExpRAG [141] provides a compelling baseline challenge: a simple *Experience Bank* of trajectory embeddings, retrieved via nearest-neighbor search and injected into the system prompt, achieves 83.6% on ALFWorld *without training*—outperforming Mem0 (33.6%), A-MEM (34.7%), Reflexion (42.7%), and Memory Bank (40.3%). Combined with LoRA fine-tuning (ExpRAG-LoRA), the same architecture reaches 94.1%, surpassing all supervised baselines. Both results call for "building stronger baselines before exploring complex engineered architectures." The key finding is that solver capability moderates the value of complex memory: weaker models (Qwen 2.5-7B) benefit up to +15pp from experience retrieval, while stronger models (Llama 3.2-3B) gain only +7pp—another instance of the model-size equalizer effect (§5.7).

Table 5
Model-size equalizer effect: graph memory improvements at small vs. large model scales. The ratio column shows how much stronger the effect is at small scale.

System	Metric	Small	Large	Ratio
GAM vs Mem0	LoCoMo F1	+39% (7B)	+7% (GPT-4o)	5.6×
GAM vs MemOS	LongDialQA F1	+86% (7B)	+5% (GPT-4o)	17×
TGM inference	Avg EM	+26% (4B)	+9% (8B)	2.8×
TGM training	Avg EM	+14% (4B)	+3% (8B)	4.1×

The counterargument: all systems plateau below 25% on *cognitive* tasks. Whether GNNs or topological methods can break this ceiling is the central open question. ENGRAM [41] offers a useful counterpoint, showing that simpler typed memory stores—separating episodic (m^{epi}), semantic (m^{sem}), and procedural (m^{pro}) records through a single router and retriever—can achieve state-of-the-art on LoCoMo and surpass the full-context baseline by 15 absolute points on LongMemEval while using only ∼1% of the tokens, challenging the trend toward architectural complexity. A systematic comparison by Zeng et al. [92] across 6 datasets reveals that knowledge triples *alone* underperform raw text chunks for most tasks—the triple decomposition sacrifices contextual nuance. However, *mixed memory* combining multiple structure types with iterative retrieval consistently wins, and among single structures, atomic facts (minimal self-contained propositions) provide the best balance between structure and context. This suggests that the value of graph-based memory lies not in replacing text with triples, but in providing *relational scaffolding* that complements rather than replaces textual representations—a reminder that graph complexity must earn its keep.

5.7. Graph Memory as Model-Size Equalizer

An underexplored dimension of graph-based memory is its interaction with model size. Evidence across our corpus reveals a consistent pattern: *graph-structured memory disproportionately benefits smaller models*, acting as a capability equalizer that narrows the gap to frontier systems. Table 5 consolidates the evidence.

The most direct evidence comes from GAM [25]. On LoCoMo, GAM achieves Average F1 of 40.00 with Qwen 2.5-7B versus 28.86 for Mem0 (+39%) and 28.86 for MemoryOS (+39%), but only 43.14 versus 40.37 for Mem0 (+7%) and 32.41 for MemoryOS (+33%) with GPT-4o-mini. On LongDialQA, the pattern is sharper: GAM surpasses MemoryOS by 86% at 7B but only 5% at GPT-4o-mini—a 17× ratio. Baselines like A-Mem and MemoryOS exhibit "performance fluctuations when scaling down from GPT-4o-mini to Llama 3.2-3B"; GAM maintains stable advantages across all four scales tested (3B, 7B, 14B, GPT-4o-mini).

The trainable graph memory [39] provides the strongest equalization result: on inference, Qwen3-4B with graph memory achieves +25.8% improvement (vs. +9.3% for Qwen3-8B); on RL training, the improvement is +13.6% at 4B versus +3.3% at 8B. Most strikingly, *the trained Qwen3-4B with graph memory (0.426 EM) outperforms the baseline Qwen3-8B (0.395)*—graph structure directly substitutes for model scale.

GNN-RAG [10] provides a complementary result: a fine-tuned 7B model with GNN-based graph retrieval *matches GPT-4* on knowledge graph QA, outperforming LLM-based retrieval by 8.9–15.5% F1 while using 9× fewer KG tokens. MemoGraph [40] achieves 86.9% on MATH with a 7B model augmented by RGCN episodic memory. CraniMem [21], evaluated on Qwen 2.5-7B, shows the smallest performance drop under noisy input compared to both vanilla RAG and Mem0.

Four mechanisms explain why smaller models benefit more:

1. **Context window relief.** Smaller models have tighter context windows and are more susceptible to the lost-in-the-middle effect [58, 59]. Graph retrieval pre-selects relevant subgraphs, delivering focused context rather than flooding the window.
2. **Explicit structure offloads reasoning.** Larger models can sometimes infer relations from

Table 6
Retrieval efficiency comparison on LoCoMo (Qwen 2.5-7B). Tokens/Q = average tokens per query sent to the LLM.

System	Tokens/Q	Latency (s)	Avg F1
GAM	1,370	0.80	40.00
Mem0	1,534	0.51	35.38
A-Mem	4,221	2.21	24.20
MemoryOS	3,400	154.22	28.86
MAGMA	—	1.47	0.700*

*Judge score (different metric); lowest graph-method latency.

unstructured text; smaller models cannot. Graph edges make relations explicit, shifting reasoning from parametric capacity to structural representation.

3. **Token efficiency.** GNN-RAG's 9× token reduction is the difference between fitting the problem in a 32K window and not. Graph structure compresses context without information loss.
4. **Strategic compensation.** The trainable graph memory result reveals a fourth mechanism: meta-cognitive strategies extracted from graph structure compensate for limited reasoning capacity. The 4B model doesn't just get better context—it gets explicit decision-making guidance that larger models can derive implicitly.

This equalizer effect has practical implications: if graph memory can bring 7B models to frontier-level performance on structured reasoning, it enables deployment at dramatically lower cost and latency (∼14 GB vs 140 GB+ VRAM, ∼10× inference speedup). The strongest graph architectures—hierarchical and multi-graph composites—may prove most valuable not for augmenting already-capable frontier models, but for *elevating small models* to cognitive competence.

Efficiency profile. Graph-structured memory also demonstrates favorable efficiency characteristics. Table 6 compares retrieval efficiency across systems where data is available. GAM achieves the best accuracy-per-token ratio, delivering higher F1 with fewer tokens than any baseline. MemoryOS, despite its multi-layer architecture, incurs 192× the latency of GAM while achieving 28% lower F1. These results suggest that hierarchical graph structures provide both accuracy *and* efficiency gains—the graph pre-selects relevant substructures, reducing both the tokens fed to the LLM and the number of retrieval iterations. GAM's scaling analysis confirms this: as sessions grow from 3 to 27, graph edges increase from 371 to 3,307, yet tokens per query remain stable (1,604→1,836) and latency grows only modestly (3.49 s→4.71 s)—the topic layer prunes the search space before event-level traversal.

6. Memory Evolution: Learning Over Time

Graph-based memory must evolve as agents accumulate experience. The survey by Yang et al. [7] identifies two complementary evolution paradigms: *internal self-evolving* (introspective refinement without external input) and *external self-exploration* (grounding evolution in environmental feedback).

6.1. Internal Self-Evolving

Three mechanisms support introspective refinement:

Memory consolidation. Agents abstract raw experience into generalized knowledge. GAM [25] consolidates semantically complete event segments into topic nodes via LLM-based boundary detection. Zep [14] invalidates stale edges rather than deleting them, preserving provenance. Mem0 [1] deduplicates and merges similar memories. FLEX [62] organizes experiences into a hierarchical tree with "golden zones" (successful trajectories) and "warning zones" (failures), employing LLM-based

semantic gating to evaluate whether new trajectories should be integrated. FLEX demonstrates two remarkable properties: (1) *scaling laws* where performance improves log-linearly with experience library size, and (2) *cross-model experience inheritance* where experiences collected by GPT-4 transfer to and improve GPT-3.5—suggesting that memory graphs built by capable models can elevate smaller ones, complementing the model-size equalizer effect (§5.7).

Graph reasoning. Agents actively scan their memory graphs to discover latent connections. *Latent link prediction* infers transitive edges (if $A \xrightarrow{cause} B$ and $B \xrightarrow{cause} C$, insert $A \xrightarrow{leads\_to} C$). *Inductive knowledge enrichment* derives new attributes from path patterns, mitigating incompleteness in the initial memory construction.

Graph reorganization. As memory accumulates, retrieval latency grows. Systems implement forgetting mechanisms: MemoryBank [76] applies Ebbinghaus-curve forgetting ($R(t) = e^{-t/S}$) with significance-based pruning removing rarely accessed nodes. MemGPT [2] implements similar significance-based pruning within its OS-style memory management. *Topology optimization* restructures the graph by shortening paths between frequently co-retrieved concepts and creating shortcuts between correlated nodes [7]. The survey identifies scalability as a key gap: "memory systems face computational bottlenecks, with graph operations exhibiting quadratic or worse complexity," calling for memory compression, incremental update algorithms, and hardware acceleration via specialized graph processing units. Inhar [113] argues that the central question is not *whether* to compress, but *where*: monolithic compression introduces *interference*—measurable behavioral drift $\Delta_t(Q) = \mathbb{E}_{q\sim Q}[D(\pi_t(\cdot|q)\|\pi_{t+1}(\cdot|q))]$ on previously supported queries. A formal bound shows $\Delta_t(Q) \leq \rho_t \epsilon_t$ where ρ_t is retrieval-update overlap. Modular memory with local compression isolation, sparse routing, and explicit composition interfaces drives $\rho_t \ll 1$, enabling aggressive compression within modules without destabilizing unrelated behaviors. A-MAC (Adaptive Memory Admission Control) [108] addresses quality *before* storage via a gating mechanism with five interpretable value signals: $S(m) = w_1 U(m) + w_2 C(m) + w_3 N(m) + w_4 R(m) + w_5 T(m)$, where U is utility, C confidence, N novelty, R recency, and T content-type prior. Memories scoring below a learned threshold are rejected at admission time. On LoCoMo, A-MAC achieves F1 = 0.583 with 31% lower latency than unfiltered baselines; ablation reveals the content-type prior T as the single most influential factor, suggesting that *what kind* of memory to store matters more than any individual quality signal.

6.2. External Self-Exploration

Feedback-driven adaptation. Memory-R1 [61] treats memory operations (add, delete, retrieve) as learnable actions within an RL framework, receiving task-specific rewards. ExpeL [63] crystallizes successful trajectories into reusable skills while failed ones undergo comparative analysis to isolate causal errors. Experiential Reflective Learning (ERL) [106] improves on ExpeL by reflecting on *single* trajectories to generate structured heuristics (trigger conditions + actionable guidelines), then retrieving the top-k heuristics for new tasks via LLM-based scoring. On Gaia2, ERL achieves 56.1% success (+7.8% over ReAct, +5.2% over ExpeL); critically, distilled heuristics outperform raw trajectory prompting by 9.7 points, confirming that *abstraction* is essential for experiential transfer. JitRL [111] takes a fundamentally different approach: rather than distilling experience into text, it stores raw (s_t, a_t, G_t) transition triplets as a non-parametric memory bank and performs *test-time policy optimization* by retrieving relevant trajectories to estimate action advantages $\hat{A}(s,a) = \hat{Q}(s,a) - \hat{V}(s)$, then adjusting LLM logits via the closed-form update $z' = z + \beta\hat{A}$—provably the exact solution to the KL-constrained policy optimization objective. On WebArena and Jericho, JitRL sets a new state of the art among training-free methods while costing $>30\times$ less than conventional RL, demonstrating that memory-based advantage estimation can substitute for gradient updates entirely. Distilling Feedback into Memory-as-a-Tool [114] amortizes inference-time reasoning by converting transient evaluator critiques into persistent, retrievable guidelines via file-based tool calls (ls, read, write). The agent consolidates specific feedback into generalizable

principles ($M' \sim p(M'|M, \text{feed}_i, x_i)$), achieving parity with compute-heavy self-critique pipelines after just two feedback rounds on the Rubric Feedback Bench. In long-horizon mixed-task experiments (H=12), the memory agent scores 0.78 ± 0.10 versus 0.52 ± 0.25 for the baseline, demonstrating cross-domain consolidation. Memento [122] formalizes continual adaptation without LLM fine-tuning as a Memory-augmented Markov Decision Process (MA-MDP), enabling memory-based online RL where the agent learns *what to store* via reward signals—achieving low-cost continual adaptation that bridges the gap between rigid handcrafted workflows and expensive gradient updates. MEM1 [123] synergizes memory and reasoning via end-to-end RL, using GRPO to jointly train memory management and retrieval within a single framework. The key insight is that memory operations should be learned as part of the reasoning policy rather than treated as a separate preprocessing stage. NEMORI [124] addresses the fundamental question of *what deserves memory* by casting future utility assessment as a predictability problem: experiences whose outcomes are predictable from existing memory are redundant, while unpredictable ones carry novel information worth retaining—an information-theoretic alternative to handcrafted importance heuristics. The trainable graph memory [39] applies REINFORCE to calibrate edge weights based on counterfactual reward gaps.

Active inquiry. ProMem [64] enables agents to self-reflect on missing nodes or ambiguous edges, generating specific queries to fill knowledge gaps—transitioning from passive recording to proactive construction. Kimi K2.5 [65] introduces a scalable swarm mechanism, spawning parallel sub-agents to explore disjoint branches of a problem space and rapidly populate the central memory. AgentEvolver [82] autonomously generates sub-tasks to explore unexplored states, systematically filling gaps in the memory graph through self-directed exploration. InfMem [135] instantiates System-2 cognitive control for bounded-memory agents via a PreThink-Retrieve-Write loop: the agent monitors whether accumulated memory suffices to answer a question, actively retrieves bridging evidence when gaps are detected, and applies evidence-aware joint compression under a fixed overwrite budget. Trained via SFT→RL with verifier-based rewards, InfMem improves accuracy by +10–12 points over MemAgent on 1M-token QA benchmarks while reducing inference time by 3.9× via adaptive early stopping—demonstrating that active memory control outperforms passive streaming.

MemEvolve [83] goes further with meta-evolution: the memory *architecture itself* adapts through feedback-driven evolution, dynamically adjusting storage structures and indexing strategies—a second-order evolution mechanism where not just memory content but memory organization co-evolves with the agent's experience. ALMA [143] pushes this idea to its logical extreme via *meta-learning of memory designs*: a code-based search space defines memory architectures as programs, and an LLM-driven exploration loop generates, evaluates, and refines memory designs across four agent domains (dialogue, embodied, web, tool-use). The best ALMA-discovered architectures consistently surpass human-designed baselines, demonstrating that the space of effective memory systems is larger than manually explored designs suggest.

ACE (Agentic Context Engineering) [144] addresses a complementary dimension: rather than evolving memory *structure*, it evolves memory *content*. A Generator/Reflector/Curator pipeline iteratively refines domain-specific playbooks—structured context documents that encode task strategies and domain knowledge—using incremental delta updates that prevent context collapse from full rewrites. On agentic coding benchmarks ACE achieves +10.6% improvement; on finance tasks +8.6%, confirming that curated evolving context outperforms both static prompts and unstructured experience accumulation. Yu et al. [145] demonstrate that for test-time agent learning, *data curation matters more than data quantity*: carefully selecting approximately 30% of available training trajectories matches or exceeds full-dataset performance. Their curriculum-based selection uses difficulty scoring and diversity sampling to construct compact experience banks—a result with direct implications for memory consolidation, as it suggests that aggressive filtering during memory write (discarding redundant or low-signal experiences) is not merely efficient but actively beneficial for downstream retrieval quality.

6.3. Graph Traversal as Planning

The survey [7] formalizes the agent interaction loop in POMDP terms: state s_t, action a_t, transition $s_{t+1} \sim \Psi(s_{t+1}|s_t, a_t)$, partial observation $o_t = O(s_t, h_t, Q)$, and memory update $M \leftarrow \text{Update}(M, o_t, a_t, r_t)$. Under this formulation, the memory graph serves as the agent's belief state representation. Graph traversal is explicitly planning-like: "the agent maintains a bounded working memory that records the current question, partial plan, visited nodes, and evidence collected so far" [7]—a graph-search planning algorithm with belief state (working memory), goal (query), and action space (traversal operators).

GAM [25] provides a partial decision-theoretic formulation: memory management is cast as an inference-time online decision process seeking $\pi^* = \arg\min_\pi \mathbb{E}[\sum_t (C_{enc}^{(t)} + C_{inter}^{(t)})]$ minimizing encoding cost plus expected interference between graph layers—though they approximate this via the semantic divergence indicator b_t rather than solving it as a full MDP. KG-HLM [81] makes the connection explicit, representing both the hidden world state and agent memory as RDF knowledge graphs, with RDF-star temporal qualifiers (`time_added`, `last_accessed`, `num_recalled`) on every triple. A striking result: a symbolic agent using handcrafted heuristics over the KG achieves approximately 4× higher rewards than a fine-tuned Llama-3.1-8B neural agent—suggesting that current LLMs are not yet effective at structured memory management, and that graph-based symbolic reasoning retains a significant advantage for precise state tracking.

Gupta et al. [155] formalize a previously overlooked aspect of sequential memory construction: *chunk ordering under bounded memory*. In Chain-of-Agents (CoA) frameworks, where workers sequentially process context chunks through a shared bounded memory, the order of processing determines what information survives compression. By modeling inter-chunk dependencies as a Chow–Liu tree and applying breadth-first traversal, they derive dependency-aware orderings that consistently outperform both default document order and similarity-based sorting across three long-context benchmarks—demonstrating that graph-structured *processing order* is itself a memory design decision. CAN [71] demonstrates that graph learning directly improves planning in LLM agents. DAMCS [79] uses hierarchical KGs for cooperative planning in decentralized multi-agent systems. These connections suggest that the retrieval-planning boundary in graph-memory systems is more porous than typically acknowledged: retrieval *is* planning over a structured belief state.

6.4. Evolution and the Cognitive Ceiling

The evolution dimension is critical for understanding the cognitive ceiling (below 25% on LoCoMo-Plus). Current systems excel at *factual* evolution (consolidation, pruning, deduplication) but lack *cognitive* evolution: learning new reasoning patterns from past failures, adapting retrieval strategies to individual users, and discovering emergent structure in accumulated experience. The trainable graph memory's REINFORCE-based weight optimization represents an early step toward cognitive evolution, but operates at the edge level; evolving the graph *structure* (adding node types, creating new relation categories) remains open.

7. Application Domains

Graph-based memory finds application across diverse agent domains, each imposing distinct requirements on memory structure and retrieval.

Software engineering agents. Code generation and debugging require memory that captures *structural* dependencies (file hierarchies, API call graphs, module dependencies) alongside *temporal* evolution (version history, deprecation timelines). TALM [84] introduces a dynamic tree-based architecture for code agents, using divide-and-conquer over long-term memory to handle non-linear dependencies in code generation—demonstrating that hierarchical graph structure (our Pattern 2) directly addresses the complexity of SE tasks. RepoAudit [85] explores codebases via graph traversal over file dependency

graphs, enabling accurate analysis without full-repository context. MemGovern [86] arranges external information from GitHub into a retrievable knowledge base, representing memory evolution through human experience feedback. Agent KB [126] addresses a cross-cutting limitation: frameworks like smolagents, OpenHands, and OWL accumulate valuable problem-solving experience that remains trapped within individual systems. Agent KB enables cross-domain experience transfer via a shared knowledge base, allowing agents to leverage solutions discovered by other frameworks—moving toward collective intelligence for agentic problem solving. MemGrad [98] introduces textual gradients for multi-agent SE: batched behavioral feedback from execution traces is decomposed into role-specific feedback-resolution pairs (routed to Programmer, Reviewer, Tester, Product Owner via cosine similarity), abstracted into compact textual gradients, and stored in a dual retrospective–prospective memory ($M_{ret,r}$ captures recurring failure patterns; $M_{pro,r}$ encodes forward-looking improvement strategies). Applied to AgileCoder, MemGrad improves task success and reasoning stability without model fine-tuning—demonstrating that *memory as gradient descent in language space* can drive persistent multi-agent improvement. Gloaguen et al. [142] investigate a distinct form of SE memory: repository-level context files (AGENTS.md, CLAUDE.md) now present in over 60,000 GitHub repositories. Surprisingly, across multiple coding agents and LLMs evaluated on both SWE-bench Lite and a new AGENTBENCH benchmark (138 instances from repositories with developer-written context files), LLM-generated context files result in *no improvement* in task success rates while increasing inference cost by over 20%. Developer-written files provide only marginal gains (+4% on average), with some models (Sonnet 4.5) actually losing 2.5%. The mechanism: context files encourage broader exploration (more testing, more file traversal) but also introduce unnecessary requirements that make tasks harder—suggesting that static declarative memory (AGENTS.md) is less effective than dynamic experiential memory for SE agents. SWE-Bench [87] and ToolBench serve as primary benchmarks, emphasizing process memory and the ability to retain intermediate hypotheses and failed attempts. The SE domain particularly benefits from the model-size equalizer effect (§5.7): if graph memory can elevate 7B models to frontier-level reasoning over code structure, it enables on-device coding assistants with privacy-preserving local memory.

Conversational agents. Multi-session dialogue is the primary evaluation domain for graph memory. SGMem [88] connects fragmented sessions via sentence-level graph links; TReMu [57] resolves temporal ambiguity in multi-session contexts; Reflective Memory [68] combines prospective and retrospective reasoning for personalized long-term dialogue. O-Mem [120] tackles a specific failure mode of semantic-grouping-first systems: semantically irrelevant yet critical user information (e.g., a casual mention of an allergy) gets overlooked by retrieval. O-Mem implements an omni-memory system for personalized self-evolving agents that bypasses semantic pre-grouping, instead using dynamic indexing that captures both explicit preferences and latent behavioral patterns. PersonaAgent [121] integrates community-aware GraphRAG into persona-based agents, constructing LLM-derived graph indices over user-specific knowledge to enable personalized retrieval that adapts to individual preferences. ATOD [148] introduces a benchmark and evaluation framework for *advanced task-oriented dialogue* that captures agentic capabilities absent from prior benchmarks: multi-goal concurrency, dependency management between interleaved goals, long-horizon memory across non-sequential workflows, and proactive agent behavior (asynchronous follow-up, pending-state management). ATOD's goal status trajectories (Open → Pending → Completed/Failed) formalize memory requirements that conventional session-based benchmarks cannot express. DialSim [149] stress-tests conversational memory via simulation: an agent assumes a character role in scripted multi-party dialogues from TV shows (Friends, Big Bang Theory, The Office) spanning 1,300 sessions and 352K tokens, and must answer spontaneous questions using only dialogue history. The accompanying LongDialQA dataset reveals that even models with 128K–1M token context windows and RAG capabilities score below 60%, highlighting that multi-party, multi-session comprehension remains an open problem. CloneMem [150] complements dialogue-centric benchmarks by evaluating memory over *non-conversational digital traces*—diaries, social media posts, emails—spanning 1–3 year life trajectories. Unlike LoCoMo's session-fragmented

structure, CloneMem models coherent life arcs with evolving experiences, emotions, and opinions. A striking finding: simple flat retrieval outperforms consolidation-based methods in this setting, and models frequently fall back on generic narrative templates rather than grounding responses in retrieved evidence. LoCoMo remains the gold-standard conversational benchmark, though a notable limitation is that "many interaction benchmarks lack explicit supervision for memory updates when dealing with conflicting facts" [7].

Narrative agents. Long-form generation (e.g., serialized novels) stresses memory in a domain rarely studied. Chen [138] reports deployment findings from 90 chapters of novel generation (180K+ tokens over 3 months), identifying *known-information forgetting*—where characters redundantly discover facts they already learned—as a distinct failure mode from hallucination or inconsistency. The root cause is recency-based context injection that systematically excludes mid-chapter events. A three-tier memory architecture (episodic event graph, semantic character states, per-generation working memory) with *Key Facts Injection* (extracting named entities, relationship changes, and information transfers with explicit "already knows" epistemic markers) reduces incidents by 73%. This demonstrates that tracking *what characters know* (epistemic state) matters as much as tracking *what happened* (world state).

Financial agents. Financial markets demand memory with information decay patterns, interpretability for regulatory compliance, and counterfactual reasoning for risk management. FinMem [89] tackles cognitive bottlenecks in AI traders through a layered memory structure inspired by human trading behavior, separating short-term market signals from long-term strategic patterns. FactorMiner [107] extends this to *alpha discovery*: a self-evolving agent with dual memory—a *skills library* of 60+ composable financial operators and an *experience repository* storing successful alpha patterns $\mathcal{P}_{\text{suc}}$ alongside *forbidden regions* $\mathcal{P}_{\text{for}}$ that failed validation. The Ralph Loop (retrieve $\rightarrow$ generate $\rightarrow$ evaluate $\rightarrow$ distill) iteratively refines memory: $M_{t+1} = \Psi(M_t, \tau_t)$, using successful trajectories to expand skills and failed ones to prune the search space. This experience-driven memory evolution closely mirrors the feedback-driven paradigm (§6), applied to a domain where temporal decay and regime shifts make static memory particularly fragile.

Web agents. Web browsing agents require cross-session memory to avoid repeating mistakes across sessions. WebCoach [105] introduces a model-agnostic framework with three decoupled components: a *WebCondenser* that standardizes raw navigation logs into structured summaries, an *External Memory Store* (EMS) indexing complete browsing episodes, and a *Coach* that retrieves relevant past experiences and injects task-specific advice at runtime. On WebVoyager, WebCoach raises success rates from 47% to 61% (+14 points) with a 38B model; notably, smaller base models with WebCoach achieve performance comparable to the same agent using GPT-4o—another instance of the model-size equalizer effect (§5.7). Dynamic self-experience (agents expanding their own memory store) outperforms externally seeded memories, confirming that agents learn most effectively from their own trajectories. Zhang et al. [158] address a key limitation of text-only trajectory memory: web agent trajectories are inherently *multimodal* (screenshots, DOM trees, action sequences). Their GAE-Retriever learns joint trajectory embeddings via vision-language contrastive training over the Unified Agent Trajectory Dataset (7,747 demonstrations, 82,793 states), and WebRAGent—a retrieval-augmented web agent integrating both DOM and vision observations—achieves 16–22% performance gains over non-retrieval baselines on Online-Mind2Web.

Embodied and game agents. Environment-based benchmarks (ALFWorld, ScienceWorld, Minecraft) require experiential memory under partial observability. GITM (Ghost in the Minecraft) [90] uses dictionary-based memory for spatial layouts and crafting knowledge. Optimus-1 [91] combines a hierarchical directed KG with an abstracted multimodal experience pool for long-horizon Minecraft tasks. These domains stress-test memory under continuous state change, where graph structure must evolve in real-time. MemInsight [133] proposes autonomous memory augmentation for LLM agents,

enhancing semantic data representation and retrieval to address the challenges of growing memory size and the need for principled structuring. Deshpande et al. [147] introduce *episodic safety memory* for embodied planning: a dual-store architecture maintains episodic traces M_{ep} of past near-misses and violations alongside semantic safety rules M_{sem} distilled from episodes. Before executing any plan, the agent retrieves relevant safety episodes and checks plan steps against both stores. On a suite of household robot tasks with safety constraints (fragile objects, forbidden zones, human proximity), the safety-memory agent achieves 94% safety compliance versus 52% for a memoryless baseline while maintaining 97% task success—demonstrating that episodic memory can serve not only as a performance mechanism but as a *safety* mechanism, preventing the repetition of dangerous actions without sacrificing capability.

8. Beyond Flat Graphs: Geometry and Higher-Order Structure

Standard graphs capture only pairwise relationships in Euclidean space. Real memory involves higher-order structures (a meeting involves multiple participants simultaneously) and intrinsic hierarchies (topics contain episodes contain facts) that Euclidean embeddings compress with high distortion. We argue that *geometric* representations—moving beyond flat Euclidean graphs to structures that encode curvature, hierarchy, higher-order relations, and temporal dynamics intrinsically—offer the most promising path forward.

8.1. Hypergraph Memory: Structure Without Learning

The hypergraph data structure has been adopted for agent memory. HyperGraphRAG [35] represents n-ary facts via hyperedges; HyperMem [36] builds a three-level hypergraph (topics, episodes, facts); HGMem [37] groups multi-step reasoning chains; Hyper-RAG [38] captures beyond-pairwise correlations via one-step diffusion. HyperG [130] demonstrates that hypergraph representations also benefit structured knowledge comprehension beyond memory: by modeling n-ary relationships in structured data (HTML, tables), HyperG shows that LLMs can leverage higher-order structure for downstream tasks. Crucially, all these systems treat the hypergraph as a *data structure* with LLM-driven reasoning. None applies hypergraph neural networks (AllSet, UniGCN, HMPNN) or any form of learned higher-order message passing.

8.2. The Geometric Perspective

What unifies the structural approaches discussed in this section—hypergraphs, curvature, hyperbolic embeddings, Lie group manifolds, continuous-time dynamics—is a shift from *flat, Euclidean* graph representations to *geometric* ones that encode structural properties intrinsically. Topological deep learning (TDL) [48, 49] contributes richer combinatorial domains to this perspective: **simplicial complexes** for hierarchical containment, **cell complexes** [50] for flexible higher-order neighborhoods, and **sheaf neural networks** [54] that assign vector spaces with restriction maps—enabling entity semantics to vary by relationship context. These structures can alleviate GNN oversquashing [52], though no agent memory system has adopted them yet.

The actionable frontier, however, lies in **concrete geometric representations** where adjacent work already demonstrates gains on tasks closely related to agent memory. The following subsections develop three such directions: discrete curvature, hyperbolic embeddings, and continuous-time dynamics on graphs.

8.3. Curvature as a Structural Signal

Discrete Ricci curvature [162] provides a local geometric characterization of graph structure: *positively* curved edges connect nodes within tightly knit clusters, while *negatively* curved edges form bridges between communities—precisely the bottlenecks where information flow is lost. Topping et al. [162]

showed that negatively curved edges cause GNN oversquashing and introduced *Stochastic Discrete Ricci Flow* (SDRF), a rewiring algorithm that adds edges near the most negatively curved bottlenecks, earning an ICLR 2022 Outstanding Paper honorable mention. Subsequent work revisited both oversquashing and oversmoothing through Ollivier-Ricci curvature [163], establishing curvature as a unifying diagnostic for GNN pathologies.

For agent memory, the implications are direct:

1. **Bottleneck detection**: Negatively curved edges in a memory graph identify structural bottlenecks where retrieval information is lost during multi-hop traversal—the same failure mode responsible for the cognitive accuracy ceiling below 25%. Curvature-guided rewiring could add bridging edges precisely where retrieval degrades.
2. **Memory consolidation via Ricci flow**: Discrete Ricci flow iteratively deforms edge weights until communities separate naturally, producing a curvature-uniform metric that reveals cluster structure without heuristic thresholds. Applied to memory graphs, this offers a principled geometric alternative to Leiden-based community detection for hierarchical consolidation.
3. **Curvature-aware message passing**: Recent work on curvature-constrained MPNNs adapts information flow based on local geometry. In memory graphs, this could allow learned retrieval to naturally attend to cluster interiors (high curvature) while cautiously bridging across communities (low curvature).

Ricci flow has been extended to hypergraphs via edge transport [164], directly connecting to the hypergraph memory structures of HyperMem and HyperGraphRAG (§8). No agent memory system has yet exploited this connection.

8.4. Hyperbolic Geometry for Hierarchical Memory

Agent memory graphs are intrinsically hierarchical: topics contain episodes, episodes contain facts, projects contain tasks. Euclidean embeddings compress such tree-like structures with high distortion—volume grows polynomially with radius, while tree branching grows exponentially. *Hyperbolic space*, with constant negative curvature, matches this growth: the volume of a ball in the Poincaré disk grows exponentially with radius, providing a natural geometry for hierarchical data.

Nickel and Kiela [165] demonstrated that Poincaré embeddings capture hierarchical relations in remarkably low dimensions. Recent work brings this to retrieval: **HyperbolicRAG** [166] projects embeddings into the Poincaré ball and fuses Euclidean and hyperbolic rankings, encoding hierarchical depth through radial position. **HypRAG** [167] develops fully hyperbolic transformers in the Lorentz model with geometry-aware pooling, achieving up to 29% gains over Euclidean baselines on RAGBench—with over 20% radial increase from general to specific concepts, a property absent in Euclidean embeddings. HELM [168] shows that token embeddings from decoder-only LLMs exhibit substantial negative Ricci curvature, suggesting that *LLM representations are already naturally hyperbolic*—the geometric match is intrinsic, not imposed.

For temporal memory, geometric approaches are equally promising: **TeAST** [169] maps temporal KG relations onto Archimedean spiral geometry, ensuring that co-temporal relations share the same timeline while all relations evolve smoothly. Extensions using **Lie group manifolds** [170] improve temporal multi-hop reasoning by 10+ points, demonstrating that temporal structure benefits from geometric representations that Euclidean space cannot provide.

These results suggest a concrete research program: embedding hierarchical memory graphs (Pattern 2) in hyperbolic space, temporal memory (Zep, MemoTime) on spiral or Lie group manifolds, and using radial position as an intrinsic measure of memory specificity—general knowledge near the origin, episodic detail near the boundary.

8.5. Continuous-Time Dynamics on Memory Graphs

A complementary geometric frontier lies in treating memory graphs not as static snapshots with discrete updates, but as *dynamical systems* evolving in continuous time. **Temporal Graph Networks**

(TGN) [45]—already noted in §4—equip each node with a temporal memory vector updated via learned message functions upon interaction, combined with a time-aware embedding module that encodes elapsed time since last access. While TGN targets link prediction on interaction graphs, its per-node continuous-time memory is conceptually analogous to what agent memory requires: smooth decay of stale knowledge, gradual consolidation of reinforced facts, and temporal interpolation between discrete conversation events.

Graph Neural ODEs [171] generalize this idea by modeling the evolution of node representations as differential equations $\mathrm{d}\mathbf{x}/\mathrm{d}t = f(\mathbf{x}, \mathbf{A}, t)$, treating GNN message passing as discretized dynamics and replacing it with adaptive ODE solvers. This enables continuous-depth models that interpolate smoothly between observations—a natural fit for memory graphs where interactions arrive at irregular intervals. Extensions to temporal knowledge graph forecasting [172] model entity and relation embeddings as continuous trajectories, predicting future facts from learned dynamics.

Crucially, these dynamical approaches compose naturally with the geometric representations discussed above. Ricci flow (§8.3) is itself a differential equation on edge weights; Graph Neural ODEs could learn analogous flows that *continuously reshape* memory graph topology based on usage patterns. Hyperbolic Neural ODEs [173] extend continuous-time dynamics to the Poincaré ball, enabling trajectories that respect hierarchical structure. Yet **no existing agent memory system exploits any of these continuous-time mechanisms**—all current architectures perform discrete insert/update/delete operations on static graph stores, missing the opportunity for principled temporal dynamics.

8.6. Bridging the Gap: From Adjacent Results to Memory Systems

The gap between geometric *graph learning* (active, with real results) and geometric *agent memory* (nonexistent) is bridgeable. Three paths are viable without per-agent supervised training:

Path 1: Curvature as a training-free diagnostic. Ollivier-Ricci curvature and persistent homology extract structural features without any training. Applied to evolving memory graphs, curvature could identify when clusters should merge (positive curvature plateau), when retrieval bottlenecks emerge (negative curvature spikes), or when dense regions become stale—providing unsupervised *diagnostics* that complement learned retrieval.

Path 2: Pre-trained geometric foundation models. ULTRA [22] demonstrates zero-shot transfer of a GNN pre-trained on diverse KGs. A geometric foundation model pre-trained on benchmark memory traces (LoCoMo, LongMemEval) in hyperbolic space could capture generic structural patterns (hierarchy depth, temporal chains, community structure) that transfer across per-user memory graphs.

Path 3: Hybrid Euclidean-hyperbolic retrieval. Following HyperbolicRAG's dual-space approach, memory retrieval could fuse Euclidean semantic similarity (for content matching) with hyperbolic distance (for hierarchical relevance), using curvature to weight the fusion—tight clusters retrieved by content, bridge nodes retrieved by structural position.

Path 4: Continuous-time memory dynamics. Rather than discrete insert/update/delete operations, memory graphs could evolve via learned ODEs: node importance decays continuously between accesses, edge weights strengthen under Ricci-flow-like dynamics when endpoints co-occur, and consolidation emerges as an attractor of the learned system. Hyperbolic Neural ODEs [173] already provide the machinery for continuous trajectories that respect hierarchical structure—the missing step is applying them to agent memory rather than benchmark graph tasks.

8.7. Open-Source Ecosystem

The practical impact of graph-memory research depends on accessible implementations. Table 7 summarizes the open-source library landscape from the survey [7]. Notably, only OpenMemory

Table 7
Open-source agent memory libraries. ✓ = supported, — = absent. Graph = graph-based storage; Temp = temporal mechanism; Hier = hierarchical structure.

Library	License	Graph	Retr.	Temp.	Hier.	Agent
Cognee	Apache 2.0	✓	✓	—	—	✓
Mem0	Apache 2.0	✓	✓	—	✓	✓
Graphiti	Apache 2.0	✓	✓	—	—	✓
OpenMemory	Apache 2.0	✓	✓	✓	✓	✓
LightMem	MIT	—	✓	✓	✓	✓
Memary	MIT	✓	✓	—	✓	—
MemMachine	MIT	✓	—	✓	—	—

supports all key capabilities simultaneously (graph storage, retrieval, lifecycle management, temporality, hierarchy, and agent integration). Most libraries lack temporal support—mirroring the research gap identified in §4. Graphiti and Cognee provide the strongest graph-native implementations, while Mem0 offers the broadest ecosystem integration.

9. Open Challenges and Research Agenda

C1: Unified temporal-geometric memory ontology. Current systems vary widely in graph types, node/edge semantics, and temporal models. A unified ontology specifying how entities, events, temporal intervals, causal relations, and higher-order groupings compose into a single structure would enable interoperability. Feng et al. [42] take a first step with a *memory transplant protocol* that disentangles architecture (retrieval strategies, gating policies, consolidation schedules) from content (the stored memory items themselves) via canonical export/import contracts and a prompt-freeze rule. Using a 2×2 factorial design across a code→math domain shift (LiveCodeBench → MATH), they find that architecture transfer is system-dependent with no universal direction, content transfer in static mode provides limited benefit, and—strikingly—weaker models benefit up to +15pp from transplantation versus +7pp for stronger ones, paralleling the model-size equalizer effect (§5.7). This is the only workshop paper addressing cross-domain memory portability.

C2: Bridging graph algorithms and learned retrieval. The retrieval spectrum (Figure 3) reveals a gap: graph algorithms dominate practice while GNN message passing is limited to two systems. RL-based memory management offers an intermediate path, but extending RL to richer decisions (which edges to prune, when to consolidate, how to merge conflicting nodes) remains open. The critical question is whether the cognitive ceiling ($<$25% on LoCoMo-Plus) can be broken by learned structural representations.

C3: Collaborative graph memory with access control. Only 7% of workshop papers address shared memory. SE teams, medical teams, and organizations require multi-user memory with access control, provenance tracking, and privacy guarantees. FalkorDB+Mem0's per-user graph isolation [15] addresses *isolation* but not *collaboration*—shared memory with fine-grained subgraph permissions remains open.

Early steps toward collaborative memory include Collabor Memory [78], which formalizes multi-agent memory sharing as dynamic bipartite access graphs $G_{UA}(t)$ (users to agents) and $G_{AR}(t)$ (agents to records), with policy-controlled read/write and provenance metadata on every record. DAMCS [79] addresses decentralized multi-agent coordination via hierarchical KGs where each agent maintains a local memory graph synchronized with a shared global graph through adaptive merging. MAS-RL [80] combines multi-agent memory with RL-based coordination, using a graph-based planner to align memory access policies across agents. MIRIX [103] introduces a dedicated multi-agent memory system for LLM-based agents with explicit inter-agent memory sharing protocols. GraphCogent [104] mitigates LLMs' working memory constraints via multi-agent collaborative graph reasoning, distributing

memory load across specialized agents that collectively maintain a shared knowledge graph. RCR-Router [127] introduces role-aware context routing for multi-agent systems, using structured memory to selectively expose relevant context to each agent based on its role—reducing token waste from redundant memory exposure while improving adaptive collaboration across interaction rounds. Decentralized generative agents [128] combine LLM-powered planning with adaptive hierarchical KGs, where each agent maintains a local KG synchronized through cooperative planning protocols—demonstrating that hierarchical graph memory naturally supports decentralized multi-agent coordination. However, all these systems treat access control as binary (shared/private); fine-grained permissions (read-only, append-only, time-limited access) and provenance tracking across organizational boundaries remain unaddressed. MINJA [94] demonstrates a concrete attack vector: *query-only memory injection* where an attacker—without direct memory access—injects malicious records by crafting bridging steps that connect benign queries to adversarial reasoning chains. Across three diverse agents (RAP, EHRAgent, QA Agent), MINJA achieves 98.2% injection success and 76.8% attack success in eliciting targeted malicious outputs. The attack exploits shared memory banks common in deployed systems (e.g., ChatGPT's "Improve the model for everyone" feature), where any user's interaction can poison future retrievals for other users. This underscores that graph-based memory's relational structure is a double-edged sword: the same multi-hop traversal that enables rich retrieval also amplifies adversarial influence through bridging paths.

The survey [7] notes that graph-based memory structures "introduce unique vulnerabilities where relational patterns may inadvertently expose private data through inference attacks," and calls for differential privacy for graph memory, federated architectures for on-device processing, and secure multi-party computation for collective experiences.

C4: Memory quality, provenance, and interpretability. The survey [7] identifies a scarcity of metrics for evaluating the intrinsic quality of memory graphs across structural, semantic, temporal, and operational dimensions. Current benchmarks measure downstream task accuracy but not memory health.

TierMem [95] demonstrates the practical cost of ignoring provenance: on LoCoMo, summary-only memory achieves 0.851 accuracy but 24.5% of errors stem from *write-before-query asymmetry*—compression performed before future queries are known discards details that later prove critical. Of these failures, 70.1% are structural (missing information or over-generalization). TierMem addresses this with provenance-linked two-tier memory: Tier-1 summaries carry provenance links ρ to immutable Tier-2 raw logs, enabling selective escalation when summaries prove insufficient. This achieves accuracy close to always-raw grounding (0.851 vs. 0.873) while reducing input tokens by 54.1% and latency by 60.7%. For graph-based memory, provenance tracking is particularly natural: edges can encode not just semantic relations but *evidence chains* linking abstractions to source material. Yuan et al. [154] introduce a diagnostic probing framework that disentangles retrieval failures from utilization failures in agent memory. In a controlled 3×3 factorial study crossing three write strategies (raw chunks, Mem0-style extraction, MemGPT-style summarization) with three retrieval methods (cosine, BM25, hybrid reranking), retrieval method dominates: accuracy spans 20 points across retrieval methods (57.1% to 77.2%) but only 3–8 points across write strategies. Raw chunked storage—requiring zero LLM calls at write time—matches or outperforms expensive lossy alternatives under all retrieval methods, suggesting that current compression pipelines discard useful context that downstream retrieval cannot recover. Failure analysis confirms: retrieval failure accounts for 11–46% of errors while utilization failure remains stable at 4–8%, establishing that *improving retrieval quality yields larger gains than increasing write-time sophistication.*

ShiftBench [115] exposes another evaluation blind spot: model rankings based on long-horizon average accuracy can *reverse* when evaluated on post-shift recovery. By injecting context pollution (m=100 off-topic turns) at session boundaries in LoCoMo and HaluMem-Long, ShiftBench defines Recovery@T—the hit rate in the first T post-shift turns. Under interruption, lexical baselines (TF-IDF) show rank reversal (Spearman $\rho = -0.30$), and per-conversation alignment drops from 0.94 to 0.70.

HSR's hierarchical retrieval recovers fastest despite weaker overall accuracy, suggesting that recovery dynamics—not just average performance—should guide memory policy selection.

StructMemEval [96] reveals a complementary gap: current benchmarks (LoCoMo, LongMemEval) test retrieval accuracy but not whether agents can *organize* memory into task-appropriate structures. Across tree-structured, state-tracking, and counting tasks, memory agents with structural organization hints significantly outperform retrieval-augmented LLMs, but agents fail to spontaneously adopt appropriate structures without explicit prompting—suggesting that graph memory systems should not only store and retrieve but actively *structure* their knowledge representations.

Developing provenance tracking, interactive visualization interfaces, and multidimensional quality criteria would improve both interpretability and trust in deployed systems.

C5: Scalability and efficiency. The survey [7] identifies scalability as a fundamental bottleneck: graph operations exhibit quadratic or worse complexity as memory accumulates. Our efficiency analysis (Table 6) shows that current systems already span a 192× latency range (GAM: 0.80 s vs. MemoryOS: 154.22 s). SwiftMem [101] directly addresses the latency bottleneck via query-aware indexing: rather than performing exhaustive retrieval across the entire memory store regardless of query characteristics, it selectively indexes based on predicted query patterns—a critical optimization as memory grows. Beyond RAG [102] argues that standard top-k similarity retrieval is fundamentally mismatched to agent memory: bounded interaction streams contain many near-duplicate or highly correlated spans, causing flat retrieval to return redundant context. Their decoupling-before-aggregation approach restructures retrieval to first separate memory components and then reassemble relevant evidence, reducing redundancy while preserving subtle distinguishing details. CoMem [146] attacks latency from a systems-architecture perspective: a *decoupled long-context memory model* runs asynchronously alongside the primary agent, performing memory indexing, compression, and retrieval in a k-step-off pipeline that overlaps memory management with generation. On SWE-Bench-Verified, CoMem achieves 1.4× latency reduction compared to synchronous memory baselines while maintaining equivalent task accuracy—demonstrating that memory operations need not be on the critical path of agent reasoning. LightMem [156] asks what the *minimal* effective memory pipeline looks like: a three-stage design (extraction, consolidation, retrieval) where only extraction requires task-specific metadata, while consolidation automatically builds a hierarchical memory tree via non-parametric agglomerative clustering and retrieval traverses this tree at multiple granularities. LightMem consistently outperforms both vanilla RAG and prior agentic memory baselines on personalization and code repository tasks, with latency on-par with the fastest embedding-based systems—suggesting that much of the complexity in current memory architectures may be unnecessary. LAG (Log-Augmented Generation) [157] takes a complementary efficiency approach: rather than compressing past reasoning into text summaries, it stores selected KV values from prior reasoning traces and retrieves them directly into the model's attention mechanism. This enables *verbatim reuse* of prior computation without extraction or distillation overhead, outperforming both reflection-based approaches and standard KV caching on knowledge-intensive and reasoning-intensive benchmarks. Apparaju and Gupta [151] study compute allocation for reasoning-intensive memory retrieval and find a striking asymmetry: re-ranking retrieved candidates yields +7.5 NDCG@10 and +21% improvement from expanding the candidate pool (k=10 → 100), while query expansion shows rapidly diminishing returns beyond lightweight models (+1.1 NDCG@10 from weak to strong), and inference-time "thinking" provides minimal benefit at either stage. The implication for memory systems: under fixed compute budgets, invest in deeper re-ranking over broader candidate pools rather than more sophisticated query reformulation. As memory graphs grow from session-scale ($\sim$100 nodes) to lifetime-scale (10^5+ nodes), approximate retrieval algorithms, memory compression, incremental update mechanisms, and potentially hardware acceleration via specialized graph processing will become critical.

C6: Empirical validation of geometric memory. We propose a staged agenda: (i) apply training-free geometric analysis (discrete curvature, persistent homology) as diagnostics; (ii) evaluate hyperbolic

vs. Euclidean embeddings for hierarchical memory retrieval; (iii) pre-train geometric foundation models on benchmark traces for zero-shot transfer; (iv) test continuous-time dynamics (Graph Neural ODEs) for memory decay and consolidation. Extending LoCoMo, LongMemEval, and MemoryAgentBench with cognitive annotations would focus evaluation on the tasks where flat graph algorithms plateau.

10. Conclusion

Graph-structured memory is emerging as the architectural backbone for agents that must reason beyond factual recall. Our synthesis of 160+ papers—78 graph-memory systems across 30 corpus papers, the 70-paper MemAgents workshop, and the comprehensive survey by Yang et al. [7]—reveals a field dominated by entity-relation KGs (60%) with a persistent temporal gap (57% lacking any temporal mechanism). We identify four established patterns plus emerging hypergraph and procedural graph patterns. Production systems confirm the practical value: multi-strategy retrieval combining graph traversal with vector search achieves 91.4% on LongMemEval, nearly doubling extraction-based approaches—though at 10–20$\times$ the latency.

Retrieval spans a rich spectrum from training-free graph algorithms through RL-based optimization (with at least five distinct RL formulations: edge weights, memory operations, graph construction, Q-value selection, end-to-end reasoning) to GNN message passing (still only 4 systems) and the largely unexplored geometric frontier. Post-retrieval verification via subgraph simulation represents an underexplored enhancement strategy. Neuroscience-inspired mechanisms —Hebbian strengthening, Ebbinghaus forgetting, recency decay, CLS-based reconstructive consolidation (MIRROR), Bayesian belief maintenance (Belief Engine), surprise-gated episodic writing (ReSuME), thermodynamic retrieval arbitration (MARTA), and twelve-principle cognitive architectures—provide implicit temporal grounding complementary to formal validity intervals. A fundamental impossibility result establishes that private working memory is *structurally necessary* for interactive agents: no public-only chat agent can simultaneously maintain secrecy and consistency in Private State Interactive Tasks. Memory evolution—how graphs adapt through consolidation, reasoning, and reorganization—represents an equally critical dimension, with promising advances in System-2 active control (InfMem), recursive skill co-evolution (SkillRL), test-time advantage estimation (JitRL), information-theoretic admission control (NEMORI, A-MAC), meta-learned memory architectures (ALMA), and evolving context playbooks (ACE).

The *model-size equalizer effect* is a particularly actionable finding: graph-structured memory disproportionately benefits smaller models. GAM's improvements narrow from +86% at 7B to +5% at GPT-4o-mini (17$\times$ ratio); the trainable graph memory's trained 4B model outperforms the baseline 8B. Four mechanisms explain this: context window relief, explicit structure offloading reasoning, token efficiency, and strategic compensation. If graph memory can bring 7B models to frontier-level performance, it enables deployment at $\sim$10$\times$ lower cost.

Multi-agent memory coordination, supported by early systems like Collabor Memory, DAMCS, MIRIX, and GraphCogent, remains largely unsolved—particularly fine-grained access control, differential privacy, and cross-boundary provenance tracking. Security is an underappreciated concern: MINJA demonstrates that query-only interaction can inject malicious records with 98% success, exploiting the same relational structure that enables rich retrieval. Provenance tracking (TierMem), active memory structuring (StructMemEval), thermodynamic consolidation (Entropic Memory), and admission-time quality gating (A-MAC) represent promising directions for building trustworthy memory systems. Diagnostic analysis reveals that retrieval quality—not write-time sophistication—is the primary performance bottleneck (20-point accuracy spread vs. 3–8 points for write strategies), while curriculum-based data curation shows that $\sim$30% of training experiences suffice for full performance, suggesting aggressive memory filtering is actively beneficial. Domain-specific memory evolution, exemplified by FactorMiner's dual skill-experience architecture for financial alpha discovery, shows how experience-driven feedback loops can specialize graph memory for high-stakes domains. Episodic safety memory achieves 94% compliance vs. 52% baseline while maintaining 97% task success, demonstrating that memory serves not only as a performance but also as a *safety* mechanism. Epistemic tracking

for narrative agents demonstrates that memory must encode not just *what happened* but *who knows what*—reducing known-information forgetting by 73%. SimpleMem's Pareto-optimal compression (+26.4% F1, 30× token reduction) and modular compression theory (interference $\leq$ retrieval-update overlap) point toward principled memory efficiency. A formal result that tool-augmented retrieval provably scales beyond in-weight memorization provides theoretical grounding for the entire external memory paradigm. New benchmarks extend evaluation beyond dialogue: AMA-Bench tests memory over agent-environment trajectories across six domains, CloneMem evaluates life-trajectory memory from non-conversational digital traces, ATOD benchmarks multi-goal agentic dialogue with dependency management, and DialSim reveals that even 1M-token context models score below 60% on multi-party dialogue. The open-source ecosystem is maturing (11+ libraries) but still lacks comprehensive temporal support, mirroring the research gap.

The progression from flat extraction to knowledge graphs mirrors the progression from factual recall to cognitive continuity. Current systems achieve over 90% on factual retrieval but plateau below 25% on cognitive tasks. We argue that the most promising path to breaking this ceiling lies in *geometric graph representations*: discrete Ricci curvature for identifying and resolving retrieval bottlenecks, hyperbolic embeddings for hierarchical memory where adjacent work already achieves up to 29% retrieval gains, Lie group manifolds for temporal reasoning, and continuous-time dynamics (Graph Neural ODEs, hyperbolic flows) that could replace discrete memory operations with principled temporal evolution. The mathematical tools exist; the agent memory application does not yet. Bridging this gap—connecting the geometry and dynamics of graph structure to the cognition of memory-augmented agents—is the defining open frontier.

References

[1] Chhikara, P., et al.: Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In: ECAI 2025
[2] Packer, C., et al.: MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 (2023)
[3] Gurawaliya, H., Nalenz, M., Ma, M.: Agentic Memory Realized: From Stateless Models to Cognitive Continuity. appliedAI Initiative (2026)
[4] Hu, Y., et al.: MemoryAgentBench: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In: ICLR 2026
[5] Wu, D., et al.: LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In: ICLR 2025
[6] Vectorize: Best AI Agent Memory Systems—Architecture, Benchmarks, and Trade-offs. Vectorize.io (2026). https://vectorize.io/articles/best-ai-agent-memory-systems
[7] Yang, C., et al.: Graph-based Agent Memory: Taxonomy, Techniques, and Applications. arXiv:2602.05665 (2026)
[8] MemAgents: Memory for LLM-Based Agentic Systems. ICLR 2026 Workshop
[9] Gutierrez, B.J., et al.: HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In: NeurIPS 2024
[10] Mavromatis, C., Karypis, G.: GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning. In: ACL 2025 Findings
[11] Sumers, T.R., Yao, S., Narasimhan, K., Griffiths, T.L.: Cognitive Architectures for Language Agents. In: TMLR (2024)
[12] Hu, Y., et al.: Memory in the Age of AI Agents: A Survey — Forms, Functions and Dynamics. arXiv:2512.13564 (2025)
[13] Banf, M., et al.: Memory-Aware Software Engineering Agents: From Stateless Tools to Learning Collaborators. In: AI4SE Workshop, KI 2026 (2026)
[14] Rasmussen, P., et al.: Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 (2025)

[15] FalkorDB: Graph Memory for LLM Agents with Mem0 and FalkorDB. FalkorDB Technical Blog (2025). https://www.falkordb.com/blog/graph-memory-llm-agents-mem0-falkordb/
[16] Hindsight: Open-Source AI Memory System with Multi-Strategy Retrieval. https://github.com/pchaganti/bx-hindsight (2025)
[17] Neo4j Labs: Agent Memory—A Graph-Native Memory System for AI Agents and Context Graphs. https://github.com/neo4j-labs/agent-memory (2026)
[18] Edge, D., et al.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130 (2024)
[19] Guo, Z., et al.: LightRAG: Simple and Fast Retrieval-Augmented Generation. In: EMNLP 2025
[20] Chen, B., et al.: PathRAG: Pruning Graph-based Retrieval Augmented Generation with Relational Paths. arXiv:2502.14902 (2025)
[21] CraniMem: Cranial-Inspired Gated and Bounded Memory for Agentic Systems. In: MemAgents Workshop, ICLR 2026
[22] UltRAG: A Universal Simple Scalable Recipe for Knowledge Graph RAG. In: MemAgents Workshop, ICLR 2026
[23] Anokhin, P., et al.: AriGraph: Learning Knowledge Graph World Models with Episodic Memory for LLM Agents. In: IJCAI 2025
[24] Modarressi, A., et al.: MemLLM: Finetuning LLMs to Use an Explicit Read-Write Memory. In: ICLR 2025
[25] Wu, Z., et al.: GAM: Hierarchical Graph Memory for LLM-based Agents. In: MemAgents Workshop, ICLR 2026
[26] G-Memory: Hierarchical Graph Memory for LLM Agents in Multi-Turn Interaction. In: NeurIPS 2025
[27] MemFly: On-the-Fly Memory Optimization via Information Bottleneck. In: MemAgents Workshop, ICLR 2026
[28] Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents. In: MemAgents Workshop, ICLR 2026
[29] Fast-Write, Deep-Read: EcphoryRAG as Dynamic Associative Memory for Lifelong Agents. In: MemAgents Workshop, ICLR 2026
[30] Wu, S., et al.: EMG-RAG: Retrieval-Augmented Generation over Editable Memory Graphs for Personalized LLM Agents. In: EMNLP 2024
[31] LP-RAG: A Link Prediction-Based Framework for Retrieval-Augmented Generation. In: MemAgents Workshop, ICLR 2026
[32] MAGMA: A Multi-Graph based Agentic Memory Architecture for AI Agents. arXiv:2601.03236 (2026)
[33] Shibui, Y.: Organizing Documents with Temporal, Multimodal, Multigraph Structures and Agentic RAG. (2026)
[34] AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications. In: MemAgents Workshop, ICLR 2026
[35] HyperGraphRAG: Hypergraph-Driven Retrieval-Augmented Generation for n-ary Relational Knowledge. In: NeurIPS 2025
[36] HyperMem: Three-Level Hypergraph Memory for LLM Agents. arXiv:2604.08256 (2026)
[37] Zhou, C., et al.: Improving Multi-step RAG with Hypergraph-based Memory for Long-Context Complex Relational Modeling. arXiv:2512.23959 (2025)
[38] Hyper-RAG: Improving Retrieval-Augmented Generation with Hypergraph Structures. In: Nature Communications (2026)
[39] From Experience to Strategy: A Trainable Agent Memory with Heterogeneous Graph Memory. arXiv:2511.07800 (2025)
[40] MemoGraph: Augmenting LLMs with Explicit Episodic Memory for Multi-Step Mathematical Reasoning. In: MemAgents Workshop, ICLR 2026
[41] ENGRAM: Typed Memory for LLM Agents. arXiv:2511.12960 (2025)
[42] Memory Transplants for LLM Agents: Disentangling Architecture and Content Transfer. In:

MemAgents Workshop, ICLR 2026
[43] Multi-Agent Collaborative Framework for Intelligent IT Operations. In: MemAgents Workshop, ICLR 2026
[44] Mitigating Heterogeneity among Factor Tensors via Lie Group Manifolds for TKG Embedding. In: NAACL 2025
[45] Rossi, E., et al.: Temporal Graph Networks for Deep Learning on Dynamic Graphs. arXiv:2006.10637 (2020)
[46] Temporal Reasoning over Evolving Knowledge Graphs. arXiv:2509.15464 (2025)
[47] Wang, C., et al.: LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-based Code Completion. In: ICSE 2025
[48] Papamarkou, T., et al.: Position: Topological Deep Learning is the New Frontier for Relational Learning. In: ICML 2024
[49] Papillon, M., et al.: Architectures of Topological Deep Learning: A Survey of Message-Passing Topological Neural Networks. arXiv:2304.10031 (2024)
[50] Bodnar, C., et al.: Weisfeiler and Lehman Go Cellular: CW Networks. In: NeurIPS 2021
[51] Copresheaf Topological Neural Networks: A Generalized Deep Learning Framework. arXiv:2505.21251 (2025)
[52] Demystifying Topological Message-Passing with Relational Structures. In: ICLR 2025
[53] Topological Blindspots: Understanding and Extending Topological Deep Learning Through the Lens of Expressivity. In: ICLR 2025
[54] Heterogeneous Sheaf Neural Networks. arXiv:2409.08036 (2024)
[55] D-RAG: Differentiable Retrieval-Augmented Generation for Knowledge Graph Question Answering. In: EMNLP 2025
[56] MemoTime: Memory-Augmented Temporal Knowledge Graph Enhanced Large Language Model Reasoning. arXiv:2510.13614 (2025)
[57] TReMu: Towards Neuro-Symbolic Temporal Reasoning for LLM-Agents with Memory in Multi-Session Dialogues. In: ACL 2025 Findings
[58] Liu, N.F., et al.: Lost in the Middle: How Language Models Use Long Contexts. In: TACL (2024)
[59] Dong, H., et al.: Lost in the Haystack: Smaller Needles are More Difficult for LLMs to Find. arXiv:2505.18148 (2026)
[60] Kynoch, B., Latapie, H., van der Sluis, D.: RecallM: An Adaptable Memory Mechanism with Temporal Understanding for Large Language Models. arXiv:2307.02738 (2023)
[61] Memory-R1: Treating Memory Operations as Learnable Actions within Reinforcement Learning. arXiv (2025)
[62] FLEX: LLM-based Semantic Gating for Memory Evolution in Long-Horizon Agents. arXiv (2025)
[63] Zhao, A., et al.: ExpeL: LLM Agents Are Experiential Learners. In: ICLR 2024
[64] ProMem: Proactive Memory for LLM Agents via Self-Reflective Gap Detection. arXiv (2025)
[65] Kimi K2.5: Scalable Reinforcement Learning for LLM Agent Exploration. arXiv (2025)
[66] MAQR: Memory-Augmented Query Reconstruction for LLM Knowledge Graph Reasoning. arXiv:2503.05193 (2025)
[67] Mem-α: Memory Construction via Reinforcement Learning for LLM Agents. arXiv:2509.25911 (2025)
[68] In Prospect and Retrospect: Reflective Memory Management for Long-Term Personalized Dialogue Agents. In: ACL 2025
[69] MemSearcher: Teaching LLMs to Reason, Search and Manage Memory by End-to-End Reinforcement Learning. arXiv:2511.02805 (2025)
[70] H-MEM: Hierarchical Memory with GNN Cross-Layer Representations for Long-Term Reasoning. (2025)
[71] Can Graph Learning Improve Planning in LLM-Based Agents? In: NeurIPS 2024
[72] SimGRAG: Leveraging Similar Subgraphs for Knowledge Graph-Driven Retrieval-Augmented Generation. In: ACL 2025 Findings
[73] MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. arXiv:2509.24704 (2025)

[74] Neural Graph Memory: Hebbian-Inspired Associative Retrieval for LLM Agents. (2025)
[75] Mnemosyne: Human-Inspired Long-Term Memory Architecture for Edge-Based LLMs. arXiv:2510.08601 (2025)
[76] Zhong, W., et al.: MemoryBank: Enhancing Large Language Models with Long-Term Memory. In: AAAI 2024
[77] LiCoMemory: Lightweight and Cognitive Agentic Memory for Efficient Long-Term Reasoning. arXiv:2511.01448 (2025)
[78] Collabor Memory: Multi-Agent Collaborative Memory Sharing with Dynamic Access Control. arXiv:2505.18279 (2025)
[79] DAMCS: LLM-Powered Decentralized Agents with Adaptive Hierarchical Knowledge Graphs for Multi-Agent Coordination. arXiv:2502.05453 (2025)
[80] MAS-RL: Enhancing Multi-Agent Systems via Reinforcement Learning with Graph-Based Planner and Policy. arXiv:2503.10049 (2025)
[81] KG-HLM: Leveraging Knowledge Graph-Based Human-Like Memory Systems to Solve Partially Observable Markov Decision Processes. arXiv:2408.05861 (2025)
[82] AgentEvolver: Self-Evolving Agent Exploration via Sub-Task Generation. arXiv:2511.10395 (2025)
[83] MemEvolve: Meta-Evolution of Agent Memory Systems. arXiv:2512.18746 (2025)
[84] TALM: Tree-Structured Multi-Agent Architecture for Code Generation. arXiv:2510.23010 (2025)
[85] RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing. (2025)
[86] MemGovern: Governing Code Agents through Human Experiences. (2025)
[87] Jimenez, C.E., et al.: SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In: ICLR 2024
[88] SGMem: Sentence Graph Memory for Long-Term Conversational Agents. arXiv:2509.21212 (2025)
[89] FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design. (2025)
[90] Zhu, X., et al.: Ghost in the Minecraft: Generally Capable Agents for Open-World Environments via Large Language Models with Text-Based Knowledge and Memory. arXiv:2305.17144 (2023)
[91] Optimus-1: Hybrid Multimodal Memory Empowered Agents Excel in Long-Horizon Tasks. In: NeurIPS 2024
[92] Zeng, Z., et al.: Structural Memory in LLM Agents: A Systematic Comparison. arXiv:2412.15266 (2024)
[93] Du, J., Zhao, H.: Entropic Memory: A Thermodynamics-Inspired Consolidation Mechanism for Lifelong Agent Learning. In: MemAgents Workshop, ICLR 2026
[94] Memory Injection Attacks on LLM Agents via Query-Only Interaction. In: MemAgents Workshop, ICLR 2026
[95] Zhu, Q., et al.: From Lossy to Verified: A Provenance-Aware Tiered Memory for Agents. In: MemAgents Workshop, ICLR 2026
[96] Shutova, A., Olenina, A., Vinogradov, I., Sinitsin, A.: Evaluating Memory Structure in LLM Agents. In: MemAgents Workshop, ICLR 2026
[97] Bi, D., Hu, Y., Nasir, M.N.: Real-Time Procedural Learning From Experience for AI Agents. In: MemAgents Workshop, ICLR 2026
[98] Natekar, A., Ranjan, A., Srivastava, V., Karande, S.: MemGrad: A Memory-Guided Optimization of Agentic Software Development via Abstracted Textual Gradients. In: MemAgents Workshop, ICLR 2026
[99] Zhang, K., et al.: AssoMem: Scalable Memory QA with Multi-Signal Associative Retrieval. arXiv:2510.10397 (2025)
[100] LEGOMem: Modular Procedural Memory for Multi-Agent LLM Systems for Workflow Automation. arXiv:2510.04851 (2025)
[101] Tian, A., et al.: SwiftMem: Fast Agentic Memory via Query-Aware Indexing. arXiv:2601.08160 (2026)
[102] Hu, Z., et al.: Beyond RAG for Agent Memory: Retrieval by Decoupling and Aggregation. arXiv:2602.02007 (2026)

[103] MIRIX: Multi-Agent Memory System for LLM-Based Agents. arXiv:2507.07957 (2025)
[104] GraphCogent: Mitigating LLMs' Working Memory Constraints via Multi-Agent Collaborative Graph Reasoning. arXiv:2508.12379 (2025)
[105] Liu, G., et al.: WebCoach: Self-Evolving Web Agents with Cross-Session Memory Guidance. In: MemAgents Workshop, ICLR 2026
[106] Allard, M.-A., Teinturier, A., Xing, V., Viaud, G.: Experiential Reflective Learning for Self-Improving LLM Agents. In: MemAgents Workshop, ICLR 2026
[107] FactorMiner: A Self-Evolving Agent for Alpha Mining with Skills and Experience Memory. In: MemAgents Workshop, ICLR 2026
[108] Adaptive Memory Admission Control for LLM Agents via Multi-Signal Scoring. In: MemAgents Workshop, ICLR 2026
[109] SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. In: ICLR 2026
[110] Liu, J., et al.: SimpleMem: Efficient Lifelong Memory for LLM Agents. arXiv:2601.02553 (2026)
[111] Li, Y., et al.: Just-In-Time Reinforcement Learning: Continual Learning in LLM Agents Without Gradient Updates. In: ICLR 2026
[112] K, I., Krishnan, A.: Proced-Mem: Benchmarking Procedural Memory Retrieval in Language Agents Across Domains. In: MemAgents Workshop, ICLR 2026
[113] Inhar, I.: Agentic Memory Should Localize Compression. In: ICLR 2026
[114] Gallego, V.: Distilling Feedback into Memory-as-a-Tool. In: MemAgents Workshop, ICLR 2026
[115] Zhang, T.: ShiftBench: Measuring Recovery of Agent Memory Under Distribution Shift. In: MemAgents Workshop, ICLR 2026
[116] Yang, J.C., Dailisan, D., Flechtner, M.: Belief Engine: Bayesian Memory for Configurable Opinion Dynamics in LLM Agents. In: MemAgents Workshop, ICLR 2026
[117] Gaikwad, M.: Did You Check the Right Pocket? Cost-Sensitive Store Routing for Memory-Augmented Agents. In: MemAgents Workshop, ICLR 2026
[118] MemoryVLA: Perceptual-Cognitive Memory in Vision-Language-Action Models for Robotic Manipulation. arXiv:2508.19236 (2025)
[119] M3-Agent: Seeing, Listening, Remembering, and Reasoning: A Multimodal Agent with Long-Term Memory. arXiv:2508.09736 (2025)
[120] O-Mem: Omni Memory System for Personalized, Long Horizon, Self-Evolving Agents. arXiv:2511.13593 (2025)
[121] PersonaAgent with GraphRAG: Community-Aware Knowledge Graphs for Personalized LLM. arXiv:2511.17467 (2025)
[122] Memento: Fine-tuning LLM Agents without Fine-tuning LLMs. arXiv:2508.16153 (2025)
[123] MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents. arXiv:2506.15841 (2025)
[124] What Deserves Memory: Adaptive Memory Distillation for LLM Agents. arXiv:2508.03341 (2025)
[125] KG-Agent: An Efficient Autonomous Agent Framework for Complex Reasoning over Knowledge Graph. arXiv:2402.11163 (2024)
[126] Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving. arXiv:2507.06229 (2025)
[127] RCR-Router: Efficient Role-Aware Context Routing for Multi-Agent LLM Systems with Structured Memory. arXiv:2508.04903 (2025)
[128] LLM-Powered Decentralized Generative Agents with Adaptive Hierarchical Knowledge Graph for Cooperative Planning. arXiv:2502.05453 (2025)
[129] "My agent understands me better": Integrating Dynamic Human-like Memory Recall and Consolidation in LLM-Based Agents. arXiv:2404.00573 (2024)
[130] HyperG: Hypergraph-Enhanced LLMs for Structured Knowledge. arXiv:2502.18125 (2025)
[131] Hu, Y., et al.: Memory in the Age of AI Agents. arXiv:2512.13564 (2025)
[132] AI Meets Brain: Memory Systems from Cognitive Neuroscience to Autonomous Agents. arXiv:2512.23343 (2025)
[133] MemInsight: Autonomous Memory Augmentation for LLM Agents. arXiv:2503.21760 (2025)

[134] Houliston, S., Odonnat, A., Arnal, C., Cabannes, V.: Tool Use is Provably More Scalable Than In-Weight Memory for Large Language Models. In: MemAgents Workshop, ICLR 2026
[135] Wang, X., et al.: InfMem: Learning System-2 Memory Control for Long-Context Agent. In: MemAgents Workshop, ICLR 2026
[136] Hsing, N.S.: MIRROR: Complementary Encoding and Reconstructive Consolidation for Persistent State in LLM Systems. In: MemAgents Workshop, ICLR 2026
[137] Lerma-Torres, D.C.: Human-Like Lifelong Memory: A Neuroscience-Grounded Architecture for Infinite Interaction. In: ICLR 2026
[138] Chen, X.: Epistemic Memory Failures in Long-Form Narrative Agents: A Deployment Study. In: MemAgents Workshop, ICLR 2026
[139] Ferreira, D.O.C., et al.: Episodic Memory from Compression Boundaries in Latent Representation Space. In: MemAgents Workshop, ICLR 2026
[140] Talebirad, Y., Parsaee, A., Szepesvari, C.Y., Nadiri, A., Zaiane, O.: Toward a Theory of Hierarchical Memory for Language Agents. In: ICLR 2026
[141] Ferraz, T.P., Deffayet, R., Nikoulina, V., Déjean, H., Clinchant, S.: Retrieval-Augmented LLM Agents: Learning to Learn from Experience. In: ICLR 2026
[142] Gloaguen, T., Mündler, N., Müller, M., Raychev, V., Vechev, M.: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? In: MemAgents Workshop, ICLR 2026
[143] Motwani, S.R., Pang, T., Chen, A., Song, D., Hendrycks, D.: ALMA: Meta-Learning Memory Architectures for LLM Agents. In: MemAgents Workshop, ICLR 2026
[144] Li, X., Guo, D., Zhang, Y., Wang, Y.: ACE: Agentic Context Engineering via Evolving Playbooks. In: MemAgents Workshop, ICLR 2026
[145] Yu, Z., Xiao, C., Li, B., Wang, H.: Curriculum Curation for Test-Time Agent Learning: Data Selection Matters More Than Quantity. In: MemAgents Workshop, ICLR 2026
[146] Liu, Y., Zhang, Z., Li, J., Chen, W.: CoMem: A Decoupled Long-Context Model for Memory-Augmented Agents. In: MemAgents Workshop, ICLR 2026
[147] Deshpande, A., Narasimhan, K., Bisk, Y.: Safe Robot Planning with Episodic Memory. In: MemAgents Workshop, ICLR 2026
[148] Zhang, Y., Nayyeri, H., Khaziev, R., Yilmaz, E., Tur, G., Hakkani-Tür, D., Thadakamalla, H.: ATOD: An Evaluation Framework and Benchmark for Agentic Task-Oriented Dialogue Systems. In: MemAgents Workshop, ICLR 2026
[149] Kim, J., Chay, W., Hwang, H., Kyung, D., Chung, H., Cho, E., Kwon, Y., Jo, Y., Choi, E.: DialSim: A Dialogue Simulator for Evaluating Long-Term Multi-Party Dialogue Understanding of Conversational Agents. In: MemAgents Workshop, ICLR 2026
[150] Hu, S., Zhang, Z., Wei, Y., Han, X., Tang, Z., Wang, H., Chen, R.: CloneMem: Benchmarking Long-Term Memory for AI Clones. In: ICLR 2026
[151] Apparaju, S., Gupta, N.: Compute Allocation for Reasoning-Intensive Retrieval Agents. In: ICLR 2026
[152] Baldelli, D., Parviz, A., Zouaq, A., Chandar, S.: LLMs Can't Play Hangman: On the Necessity of a Private Working Memory for Language Agents. In: ICLR 2026
[153] Das, A., Roy, I.: Look Before You Leap: Thermodynamic Arbitration of Parametric and Non-Parametric Knowledge in LLM Agents via Self-Regulating Memory Architectures. In: ICLR 2026
[154] Yuan, B., Su, Y., Yao, K.: Diagnosing Retrieval vs. Utilization Bottlenecks in LLM Agent Memory. In: ICLR 2026
[155] Gupta, N., Singh, V., Iyer, A., Shiragur, K., Grover, P., Bairi, R.B., Maiti, R., Damle, S., Gupta, S.M., Maurya, R., Vageesh, D.C.: Chow–Liu Ordering for Long-Context Reasoning in Chain-of-Agents. In: MemAgents Workshop, ICLR 2026
[156] Tan, J., Yang, L., Zhao, W., Qiu, J., Zhu, M., Murthy, R., Savarese, S., Wang, H., Heinecke, S., Xiong, C.: A Lightweight, Domain-Adaptive Memory System for LLM Agents. In: ICLR 2026
[157] Chen, P.B., Zhang, Y., Roth, D., Madden, S., Andreas, J., Cafarella, M.: Log-Augmented Generation: Scaling Test-Time Reasoning with Reusable Computation. In: MemAgents Workshop, ICLR 2026
[158] Zhang, X., Cai, S., Jiang, Z., Meng, R., Wang, Z.Z., Lu, G., Wu, Z., Shang, Y., Kong, D.: Learning

Multimodal Trajectory Representations for Web Agent Planning. In: ICLR 2026
[159] From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms. In: ICLR 2026
[160] Xiao, Y., Zhou, C., Zhang, Q., Li, B., Li, Q., Huang, X.: Reliable Reasoning Path: Distilling Effective Guidance for LLM Reasoning with Knowledge Graphs. arXiv:2506.10508 (2025)
[161] Yan, B.Y., Li, C., Qian, H., Lu, S., Liu, Z.: General Agentic Memory Via Deep Research. arXiv:2511.18423 (2025)
[162] Topping, J., Di Giovanni, F., Chamberlain, B.P., Dong, X., Bronstein, M.M.: Understanding Over-Squashing and Bottlenecks on Graphs via Curvature. In: ICLR 2022
[163] Coupette, C., Dalleiger, S., Rieck, B.: Ollivier-Ricci Curvature for Hypergraphs: A Unified Framework. In: ICLR 2023
[164] Hypergraph Clustering Using Ricci Curvature: An Edge Transport Perspective. arXiv:2412.15695 (2024)
[165] Nickel, M., Kiela, D.: Poincaré Embeddings for Learning Hierarchical Representations. In: NeurIPS 2017
[166] Cao, Y., et al.: HyperbolicRAG: Enhancing Retrieval-Augmented Generation with Hyperbolic Representations. arXiv:2511.18808 (2025)
[167] HypRAG: Hyperbolic Dense Retrieval for Retrieval-Augmented Generation. arXiv:2602.07739 (2026)
[168] HELM: Hyperbolic Large Language Models via Mixture-of-Curvature Experts. arXiv:2505.24722 (2025)
[169] Li, D., et al.: TeAST: Temporal Knowledge Graph Embedding via Archimedean Spiral Timeline. In: ACL 2023
[170] Mitigating Heterogeneity among Factor Tensors via Lie Group Manifolds for Tensor Decomposition Based Temporal Knowledge Graph Embedding. arXiv:2404.09155 (2024)
[171] A Survey on Graph Neural ODEs: Bridging Neural ODEs and Graph Neural Networks. arXiv:2406.00337 (2024)
[172] Neural ODE-based Temporal Knowledge Graph Forecasting: A Survey. Neurocomputing (2023)
[173] Hyperbolic Neural ODE: Continuous-time Dynamics on Hyperbolic Manifolds. NeurIPS Workshop on Symmetry and Geometry in Neural Representations (2023)
[174] Karpathy, A.: LLM Wiki — a personal knowledge base maintained by an LLM. GitHub Gist (2025). https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
[175] Ar9av: Obsidian-Wiki — Framework for AI agents to build and maintain a digital brain through Obsidian using the LLM Wiki pattern. GitHub (2025). https://github.com/Ar9av/obsidian-wiki
[176] AgriciDaniel: Claude-Obsidian — Persistent, compounding wiki vault based on Karpathy's LLM Wiki pattern. GitHub (2025). https://github.com/AgriciDaniel/claude-obsidian
[177] Khoj AI: Khoj — Your AI second brain. Self-hostable personal AI with Obsidian integration. GitHub (2025). https://github.com/khoj-ai/khoj
[178] nashsu: LLM Wiki — AI-powered knowledge base with Louvain community detection and 4-signal knowledge graph. GitHub (2025). https://github.com/nashsu/llm_wiki